

AURIX

Microcontroller Infineon Software Architecture

Users Manual

GPT Driver

Release V3.9

2017-02

Edition 2017-02

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2017 Infineon Technologies AG

All Rights Reserved.

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS APPLICATION NOTE IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS APPLICATION NOTE MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS APPLICATION NOTE.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (www.infineon.com).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect

CONFIDENTIAL

Date	Version	Change Description
2017-02-08	3.9	Updated based on GTM code review finding that GPT init shall only be invoked after PLL is setup. Section 6.2.2 is updated
2016-09-01	3.8	Updated Table 1 as part of JIRA 0000051047-314
2016-08-05	3.7	Updated Deviation in Section 10.1
2016-05-11	3.6	Updated based on JIRA 0000051261-231 - Updated interface diagram for GPT Driver in section 2. - Updated File structure in section 3.1 - Added configuration parameters in section 4.3 - Updated I/O mode in section 6.2
2015-10-28	3.5	Updated IO mode for API.
2015-04-29	3.4	Updated for JIRA: 0000051124-49 Updated review REV_007618
2015-04-22	3.3	Updated review comments REV_007493
2015-04-15	3.2	Updated section 6.2.2 Gpt_Init.
2015-02-20	3.1	Range modified for GptNotification(Selectable and loadable)
2014-11-24	3.0	Updated 6.2 API Functions (UTP AI00251090) and updated review comments REV_006534.
2014-08-14	2.9	Updated Chapter 10 Assumption section. Modified References section(REV_006111)
2014-07-14	2.8	Removed Chapter 10 Safety Information.
2014-05-20	2.7	Traceability changes incorporated.
2014-03-26	2.6	Changed Aurix_MC-ISAR_HE_SoftwareSafetyInformation.doc to Aurix_MC-ISAR_SoftwareSafetyInformation.doc in references section.
2014-02-28	2.5	Updated for EP.
2013-11-11	2.4	Review comments incorporated for REV_004848
2013-09-24	2.3	Updated Chapter 9 and Sec 6.1
2013-08-22	2.2	Updated to include Safety features and TC29x. Chapter 9 added for Safety Information
2013-07-10	2.1	Updated for TC26x
2013-04-02	2.0	Review REV_004331 related changes
2013-03-13	1.5	File structure updated
2012-11-15	1.4	Merged multiple AS documents
2012-07-26	1.3	Updated Bugzilla Details in sec 9.1

CONFIDENTIAL

Date	Version	Change Description
2012-06-13	1.2	Review REV_003580 related changes
2012-05-14	1.1	Code structure, Sw description & Published info sections are updated
2011-08-10	1.0	Document Improved
2011-03-08	0.1	Initial draft version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing?
Your feedback will help us to continuously improve the quality of this document.
Please send your proposal (including a reference to this document) to:

mcdocu.comments@infineon.com



Table of Contents	Page
1 Introduction	14
1.1 Scope	14
1.2 Abbreviations	14
1.3 References	15
2 GPT Driver Overview	16
2.1 Software Hardware Module Mapping	16
2.1.1 GTM	16
2.1.2 GPT	17
2.1.3 IRQ	18
2.1.4 SYSTEM CLOCK	18
2.1.4.1 Software Driver Description	18
3 File Structure	19
4 Configuration Documentation	21
4.1 Configuration Concept	21
4.1.1 Configuration Class	21
4.1.2 Configuration Variant	21
4.1.3 How to read the Configuration Class field	22
4.2 Configuration Parameter Variance	22
4.3 Container: GptDriverConfiguration	22
4.3.1 GptDevErrorDetect	22
4.3.2 GptReportWakeupSource	22
4.3.3 GptPBFixedAddress	23
4.3.4 GptDebugSupport	23
4.3.5 GptRunningInUser0Mode	24
4.3.6 GptUserModeInitApiEnable	24
4.3.7 GptUserModeDeInitApiEnable	25
4.3.8 GptUserModeRuntimeApiEnable	25

Table of Contents	Page
4.4 Container: GptConfigurationOfOptApiServices	26
4.4.1 GptVersionInfoApi	26
4.4.2 GptDelInitApi	26
4.4.3 GptTimeElapsedApi	27
4.4.4 GptTimeRemainingApi	27
4.4.5 GptEnableDisableNotificationApi	28
4.4.6 GptWakeupFunctionalityApi	28
4.5 Container: GptChannelConfigSet	29
4.5.1 Container: GptChannelConfiguration	29
4.5.1.1 GptChannelId	29
4.5.1.2 GptEnableWakeup	29
4.5.1.3 GptChannelMode	30
4.5.1.4 GptSignalType	30
4.5.1.5 GptChannelTickFrequency	31
4.5.1.6 GptChannelTickValueMax	31
4.5.1.7 GptNotification	31
4.5.1.8 GptAssignedHwUnit	32
4.5.1.9 GptClockReferencePoint	32
4.5.1.10 Container: GptWakeupConfiguration	33
4.5.1.11 GptWakeupSourceRef	33
4.5.1.12 GptChannelClkSrcRef	33
4.6 Container: CommonPublishedInformation	34
4.6.1 ArMajorVersion	34
4.6.2 ArMinorVersion	34
4.6.3 ArPatchVersion	35
4.6.4 SwMajorVersion	35
4.6.5 SwMinorVersion	35
4.6.6 SwPatchVersion	36

Table of Contents	Page
4.6.7 ModuleId	36
4.6.8 VendorId	36
4.6.9 VendorApiInfix	37
4.6.10 Release	37
4.7 Container: GptSafety	38
4.7.1 GptSafetyEnable	38
4.7.2 GptInitCheckApi	38
4.7.3 GptGetModeApi	39
5 Published parameters	39
5.1 VendorId	40
5.2 ModuleId	40
5.3 InstanceId	40
5.4 ARMajorVersion	40
5.5 ARMinorVersion	40
5.6 ARPatchVersion	41
5.7 SwMajorVersion	41
5.8 SwMinorVersion	41
5.9 SwPatchVersion	41
6 API Documentation	42
6.1 API Type Definitions	42
6.1.1 Type Definition Gpt_ConfigType	42
6.1.2 Type Definition Gpt_ChannelConfigType	42
6.1.3 Type Definition Gpt_ChannelType	43
6.1.4 Type Definition Gpt_ValueType	43
6.1.5 Type Definition Gpt_ModeType	43
6.1.6 Type Definition Gpt_EnableWakeupType	44
6.1.7 Type Definition EcuM_WakeupSourceType	44

Table of Contents		Page
6.1.8	Type Definition Gpt_ClockType	44
6.1.9	Type Definition Gpt_SignalType	45
6.1.10	Type Definition Gpt_ChannelIdentifierType	45
6.2	API Functions	45
6.2.1	Gpt_GetVersionInfo	45
6.2.2	Gpt_Init	46
6.2.3	Gpt_DeInit	47
6.2.4	Gpt_GetTimeElapsed	47
6.2.5	Gpt_GetTimeRemaining	49
6.2.6	Gpt_StartTimer	50
6.2.7	Gpt_StopTimer	52
6.2.8	Gpt_EnableNotification	53
6.2.9	Gpt_DisableNotification	54
6.2.10	Gpt_SetMode	55
6.2.11	Gpt_DisableWakeup	56
6.2.12	Gpt_EnableWakeup	57
6.2.13	Gpt_CheckWakeup	58
6.2.14	Gpt_InitCheck	58
6.2.15	Gpt_GetMode	59
6.3	Interrupt Handling	60
6.3.1	Gpt_Isr	60
6.4	Exclusive Area (EA) Information	63
7	Data Description	63
8	Error Classification	63
9	Example usage	64
9.1	Configuration of the Driver	64
9.1.1	Sample Configuration of the Driver	65

Table of Contents	Page
9.1.1.1 Driver Initialization	66
9.1.1.2 API Usages	67
9.1.2 Starting a Timer Channel	67
9.1.3 Stopping a Timer Channel	67
9.1.4 Enable Notification for a Timer Channel	67
9.1.5 Disable Notification for a Timer Channel	67
9.1.6 To Get Time Elapsed For a Timer Channel	68
9.1.7 To Get Time Remaining For a Timer Channel	68
9.1.8 To set GPT diver To Sleep Mode	68
9.1.9 To Enable/Disabled Wakeup	68
9.1.10 Getting Version Information	68
9.1.11 Deinitializing GPT Driver	69
9.1.12 To get GPT diver Mode	69
10 Limitations and Assumptions	69
10.1 Assumptions and Deviations from Software Specification	69
10.2 Limitations	70
10.2.1 Interrupt Latency	70
10.3 Hardware Features That Are Not Supported	70

List of Figures

Page

Figure 1	Software Hardware Module Mapping of GPT Driver	16
Figure 2	GTM – Hardware Block Diagram(TC27x)	17
Figure 3	Interfaces of GPT Driver	19
Figure 4	File Structure	20
Figure 5	Software limitation in minimum time (high frequency) measurements.....	70

List of Tables

Page

Table 1	GTM Unit Used for Gpt Driver	14
Table 2	Abbreviations used in this document.....	14
Table 3	File Contents.....	20
Table 4	GptDevErrorDetect.....	22
Table 5	GptReportWakeupSource	22
Table 6	GptPBFixedAddress	23
Table 7	GptDebugSupport Configuration	23
Table 8	GptRunningInUser0Mode	24
Table 9	GptUserModelInitApiEnable	24
Table 10	GptUserModeDeInitApiEnable	25
Table 11	GptUserModeRuntimeApiEnable	25
Table 12	GptVersionInfoApi	26
Table 13	GptDeInitApi.....	26
Table 14	GptTimeElapsedApi	27
Table 15	GptTimeRemainingApi	27
Table 16	GptEnableDisableNotificationApi	28
Table 17	GptWakeupFunctionalityApi	28
Table 18	GptChannelId.....	29
Table 19	GptEnableWakeup	29
Table 20	GptChannelMode	30
Table 21	GptSignalType	30
Table 22	GptChannelTickFrequency	31
Table 23	GptChannelTickValueMax	31
Table 24	GptNotification.....	31
Table 25	GptAssignedHwUnit	32
Table 26	GptClockReference.....	32
Table 27	GptWakeupSourceRef	33
Table 28	GptChannelClkSrcRef.....	33
Table 29	ArMajorVersion	34
Table 30	ArMinorVersion	34
Table 31	ArPatchVersion.....	35
Table 32	SwMajorVersion	35
Table 33	SwMinorVersion	35
Table 34	SwPatchVersion.....	36
Table 35	ModuleId	36
Table 36	VendorId.....	36
Table 37	VendorApiInfix.....	37
Table 38	Release	37
Table 39	GptSafetyEnable.....	38
Table 40	GptInitCheckApi	38
Table 41	GptGetModeApi	39
Table 42	VendorId.....	40
Table 43	ModuleId	40
Table 44	InstanceId	40
Table 45	ARMajorVersion	40
Table 46	ARMinorVersion	40
Table 47	ARPatchVersion.....	41

List of Tables

Page

Table 48	SwMajorVersion	41
Table 49	SwMinorVersion	41
Table 50	SwPatchVersion	41
Table 51	Type Definition Gpt_ConfigType	42
Table 52	Type Definition Gpt_ChannelConfigType	42
Table 53	Gpt_ChannelType	43
Table 54	Gpt_ValueType	43
Table 55	Type Definition Gpt_ModeType	43
Table 56	Type Definition Gpt_EnableWakeupType	44
Table 57	Type Definition EcuM_WakeupSourceType	44
Table 58	Gpt_ClockType	44
Table 59	Gpt_SignalType	45
Table 60	Gpt_ChannelIdentifierType	45
Table 61	Specification of API Gpt_GetVersionInfo	45
Table 62	Specification of API Gpt_Init.....	46
Table 63	Specification of API Gpt_DeInit	47
Table 64	Specification of API Gpt_GetTimeElapsed.....	47
Table 65	Specification of API Gpt_GetTimeRemaining	49
Table 66	Specification of API Gpt_StartTimer.....	50
Table 67	Specification of API Gpt_StopTimer	52
Table 68	Specification of API Gpt_EnableNotification	53
Table 69	Functional Specification - Gpt_DisableNotification.....	54
Table 70	Functional Specification - Gpt_SetMode.....	55
Table 71	Functional Specification - Gpt_DisableWakeup	56
Table 72	Functional Specification - Gpt_EnableWakeup	57
Table 73	Functional Specification - Gpt_CheckWakeup	58
Table 74	Specification of API Gpt_InitCheck	58
Table 75	Specification of API Gpt_GetMode.....	59
Table 76	GPT Interrupt Function.....	60
Table 77	Service Request Node – Hardware Resource GTM	61
Table 78	Global Data Description	63
Table 79	Error Classification.....	63
Table 80	Assumptions	69

List of Code Listings		Page
Code Listing 1	Gpt Configuration Header file – Gpt_Cfg.h.....	65
Code Listing 2	Gpt Configuration Source file – Gpt_PBCfg.c.....	66
Code Listing 3	Driver Initialization in EcuM_Init function	66
Code Listing 4	Starting a Channel.....	67
Code Listing 5	Stopping a Channel	67
Code Listing 6	Enabling Notification For a Timer Channel	67
Code Listing 7	Disabling Notification For a Timer Channel	67
Code Listing 8	To Get Time Elapsed For a Timer Channel.....	68
Code Listing 9	To Get Time Remaining For a Timer Channel	68
Code Listing 10	Set GPT Driver to SLEEP Mode.....	68
Code Listing 11	Enable/Disable Wakeup.....	68
Code Listing 12	Getting Version Information of GPT Driver	68
Code Listing 13	Deinitializing GPT Driver	69
Code Listing 14	To get GPT diver Mode	69

1 Introduction

This User Manual provides information regarding the functionality, configuration parameters and API implementation for the GPT Driver.

This User Manual is intended to help the user to get acquainted with the GPT Driver implementation for the AURIX Hardware Platform. It is the documentation that describes how to use the GPT Driver.

1.1 Scope

This document addresses the following features of the GPT Driver implementation,

- AURIX microcontroller HW peripherals used to realize the Driver
- File structure of GPT Driver
- Configuration Parameters of the Driver
- Parameters published by GPT Driver
- API Specification
- Code Snippets illustrating the common use cases.
- Limitations and Assumptions

This documentation applies to the GPT Driver Variant: Variant PB.

This document addresses the driver implementation for the following variants of AURIX Family

Some of the peripheral units/feature might not be available in all the variants. In such cases, the relevant information should be discarded for the variant where the unit/feature is not applicable. The following table gives the available units/features in the variants of AURIX Family for driver usage. All the references made to ATOM in this document is not applicable for EP devices.

Table 1 GTM Unit Used for Gpt Driver

TC Variant	GTM sub module	Sub module units	Channels per sub module
TC27x	TOM	3	16
	ATOM	5	8
TC26x	TOM	2	16
	ATOM	4	8
TC29x	TOM	5	16
	ATOM	9	8
TC21x , TC22x and TC23x	TOM	2	16
	ATOM	0	0

1.2 Abbreviations

The abbreviations used inside this document are explained in [Table 2](#).

Table 2 Abbreviations used in this document

Abbreviation	Explanation
μC	Microcontroller
API	Application Programming Interface
AUTOSAR	AUTomotive Open System Architecture
ATOM	ARU Timer Output Module

Abbreviation	Explanation
ARU	Advanced Routing Unit
BSW	Basic Software
DET	Development Error Tracer
ECU	Electronic Control Unit
GPT	General Purpose Timer
GTM	Generic Timer Module
HW	Hardware
I/O	Input/Output
ICU	Input Capture Unit
ISR	Interrupt Service Routine
MCAL	MicroController Abstraction Layer
MCU	MicroController Unit
Mux/mux	Multiplexer
RAM	Random Access Memory
SW	Software
ROM	Read Only Memory
TOM	Timer Output Module

1.3 References

This section lists down all relevant documents for this User Manual.

Some of the artifacts listed below will not contain the version number. For those artifacts, kindly refer the latest version.

- [1] Specification of GPT Driver,
AUTOSAR_SWS_GPT_Driver.pdf Release 4.0.3
- [2] Requirements of GPT Driver,
AUTOSAR_SRS_GPT_Driver.pdf Release 4.0.3
- [3] User Manual MCU Driver,
Aurix_MC-ISAR_UM_MCUDriver.pdf
- [4] User Manual GTM Driver,
Aurix_MC-ISAR_UM_GTMDriver.pdf
- [5] User Manual IRQ Driver,
Aurix_MC-ISAR_UM_IRQDriver.pdf
- [6] Target Data Sheet:
tc27xC_um_v1.5.pdf
- [7] Target TC26x User Manual: tc26x_um_V1.0.pdf
- [8] Target TC29x User Manual: tc29x_um_V1.0.pdf
- [9] Target Data Sheet: tc27xC_um_v1.5.pdf
- [10] Target user manual for TC23x and TC22x: tc23x_tc22x_um_v1.0.pdf

2 GPT Driver Overview

The GPT Driver uses the GTM peripheral of the AURIX platform. The Driver can be configured with multiple channels. Each channel consists of a timer cell for which Clock and Timeout value has to be configured. The following operations can be performed on each channel.

- Start / Stop the timer
- Avail wakeup functionality
- Enable / Disable notification
- One Shot or Continuous mode of operation

2.1 Software Hardware Module Mapping

This section introduces the user to the HW features used for implementation. The [Figure 1](#) Illustrates the AURIX HW peripherals and their interaction with the drivers.

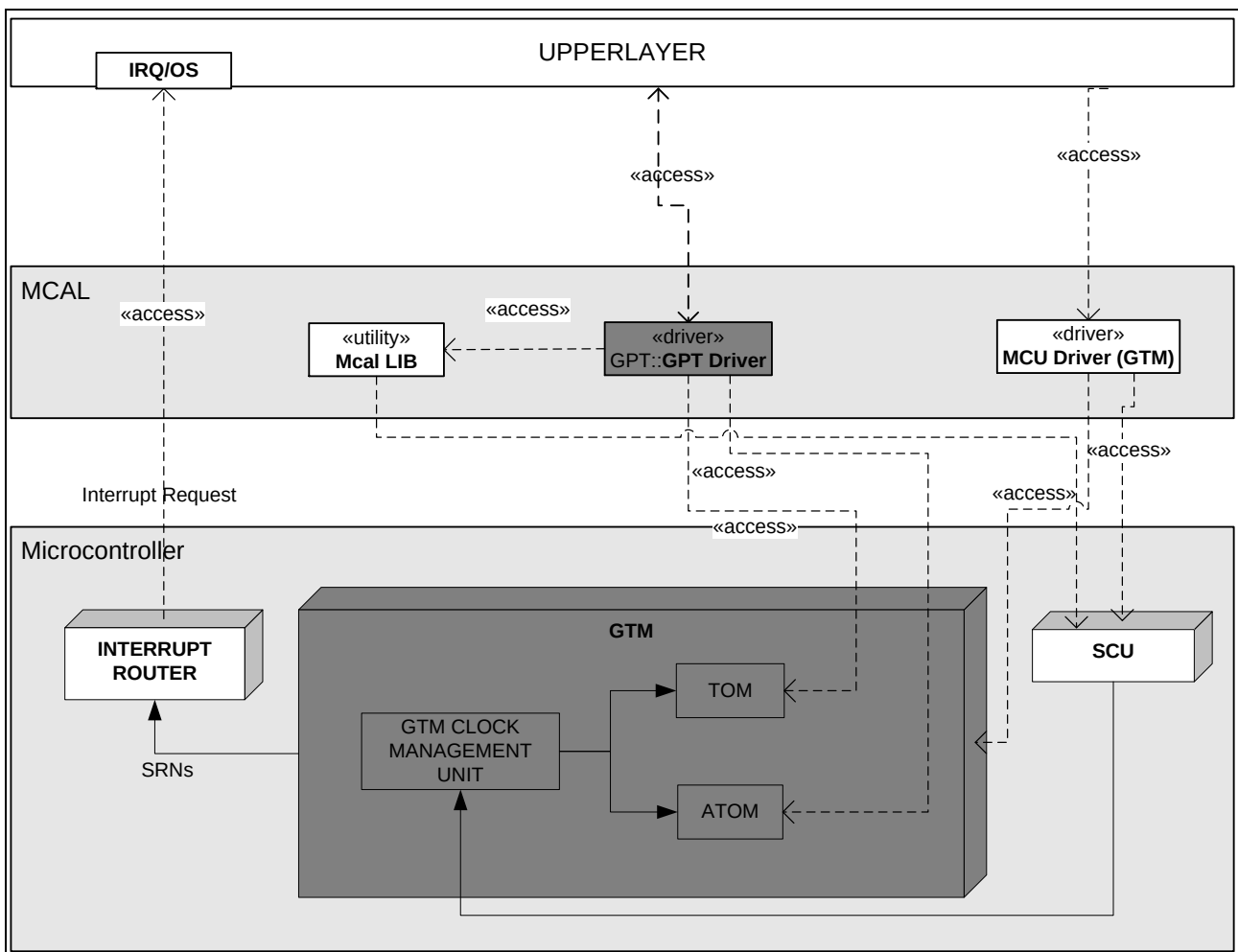


Figure 1 Software Hardware Module Mapping of GPT Driver

The different HW peripherals needed by the Driver and their purpose is explained below.

2.1.1 GTM

The GTM hardware block diagram illustrated in [Figure 2](#) is used as the basis for the hardware resources. The purpose of the chapter is only to focus on the resources used by the GPT Driver i.e. TOM/ ATOM Channels.

The use of GTM is not limited only to the GPT Driver. Additionally PWM and Complex Driver use the common resources of GTM. In order to initialize these common resources of GTM, the GTM Driver is needed. GTM Driver is also responsible for clock initialization and Input/Output MUX programming. The initialization of the common resources of the GTM is covered in detail in the User Manual of GTM[4].

To implement GPT channel, TOM/ ATOM Channels are used. It has to be ensured by proper configuration that same resources should not be used by PWM or Complex drivers.

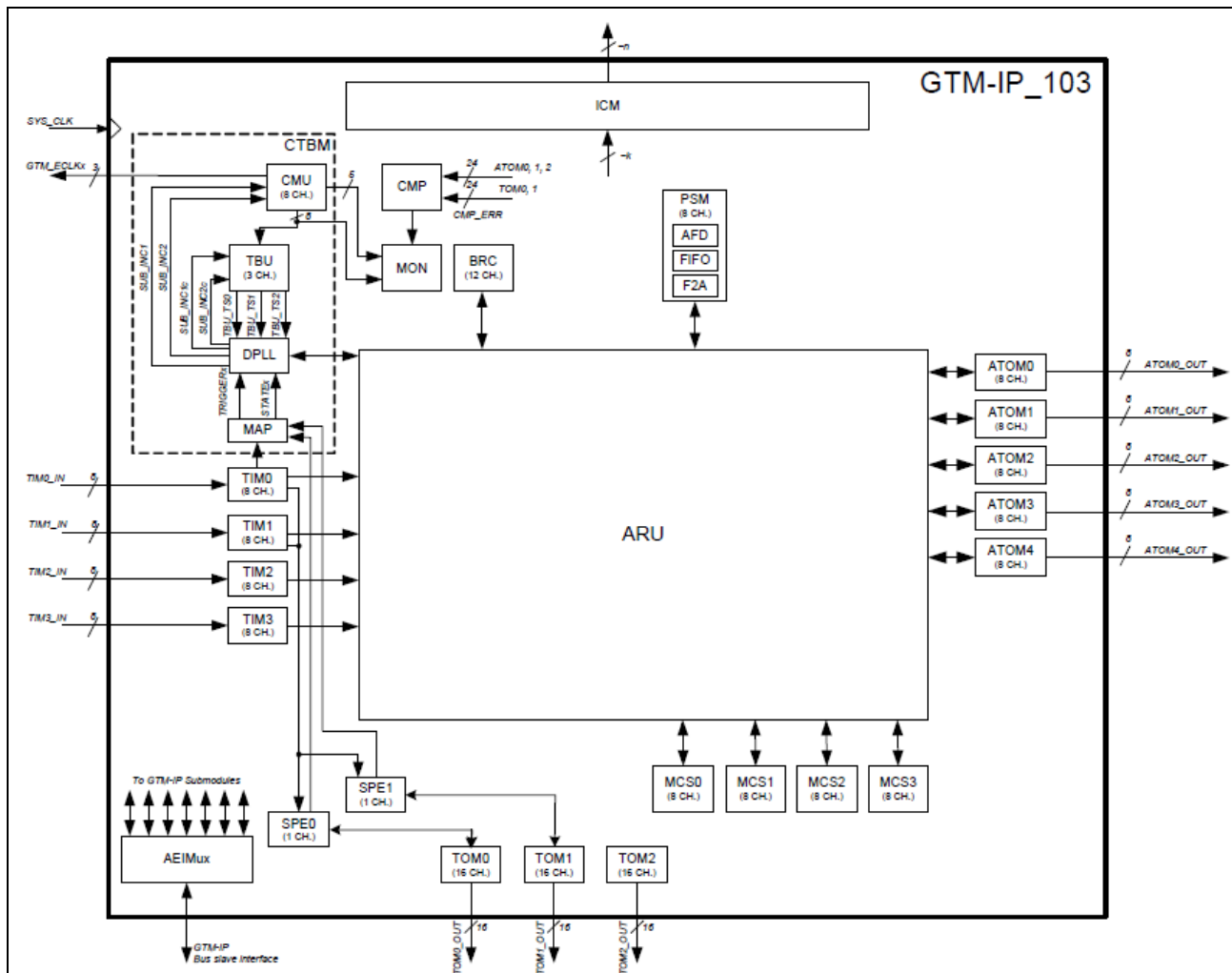


Figure 2 GTM – Hardware Block Diagram(TC27x)

2.1.2 GPT

GTM peripheral TOM/ATOM Channel resources are used to implement GPT Channel.

The width of the TOM is of 16 bits, hence the maximum timeout value can be 0xFFFF.

The width of the ATOM is of 24 bits, hence the maximum timeout value can be 0xFFFFFF.

Note: In case of GPT Channel in One Shot Mode, configure the priority for the respective TOM/ATOM channel SRN node in IRQ module, the configuration is mandatory otherwise the channel will not work properly.

2.1.3 IRQ

IRQ Driver is responsible for initializing the priority of interrupts required by GTM. The mapping of TOM/ATOM resources to respective SRN is given in [Table 77](#).

2.1.4 SYSTEM CLOCK

The system clock is set up via the MCU Driver. It is mandatory to set up an appropriate system clock. Note that the GTM peripheral clock is derived from the system clock.

See also MCU User Manual for details about MCU configuration [\[3\]](#)

The clock source for timer cell is derived from the GTM clock management unit (CMU). The CMU distributes 5 Fixed clock lines which clock the TOM channels and 8 Configurable Clock lines which clock the ATOM Channels. The clock source configuration is part of the GTM global configuration, performed in the GTM driver configuration.

See also GTM user manual for details about clock line configuration [\[4\]](#).

2.1.4.1 Software Driver Description

The [Figure 3](#) shows the Application Programming Interface (API) for the GPT Driver and its interaction with other software modules. Development Error Tracer (DET) is required if DET is enabled for the GPT Driver.

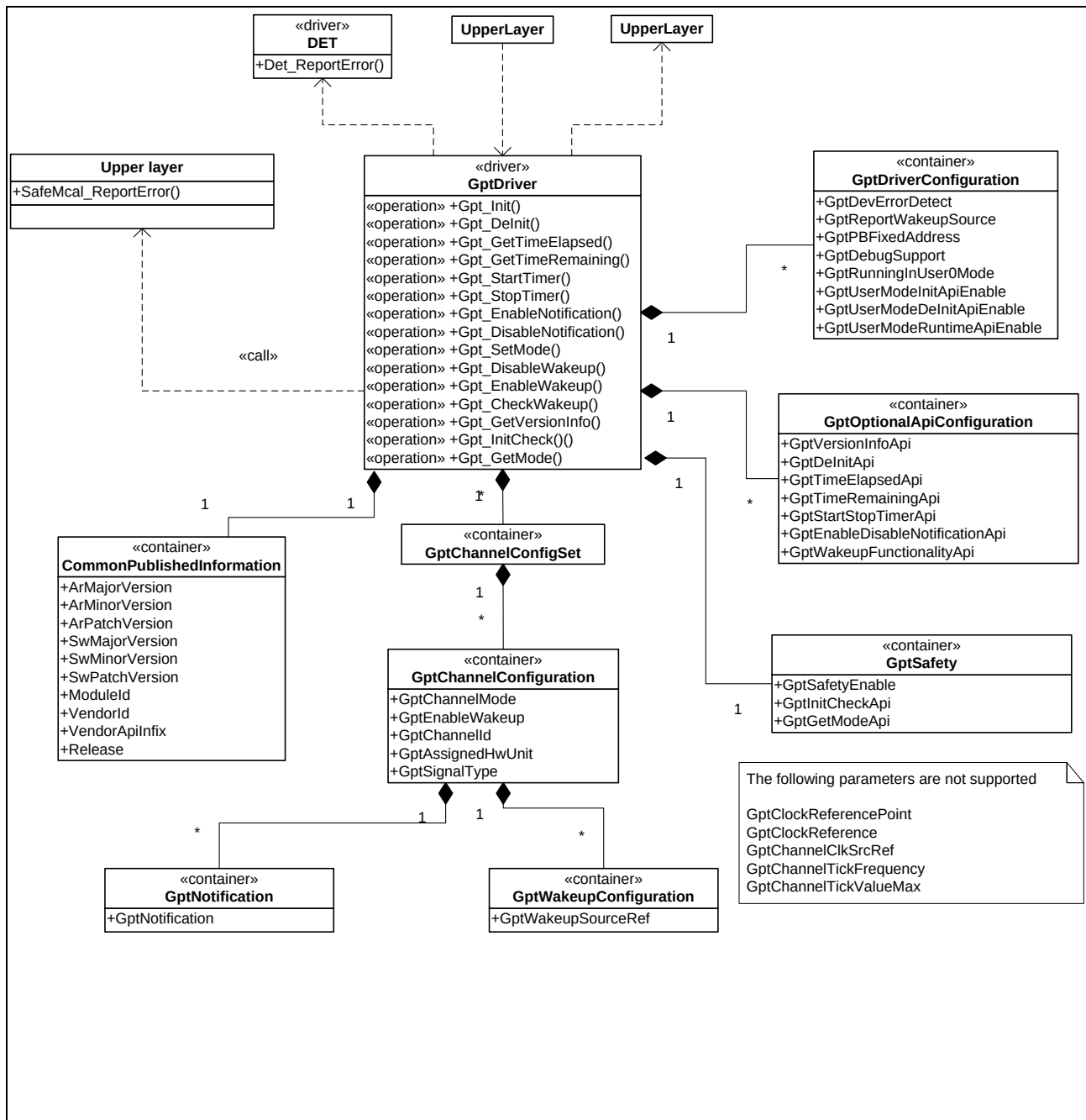


Figure 3 Interfaces of GPT Driver

The Configuration is further explained in chapter 4. The API's are explained in chapter 6.

3 File Structure

This section provides details about the GPT Driver files and other related files.

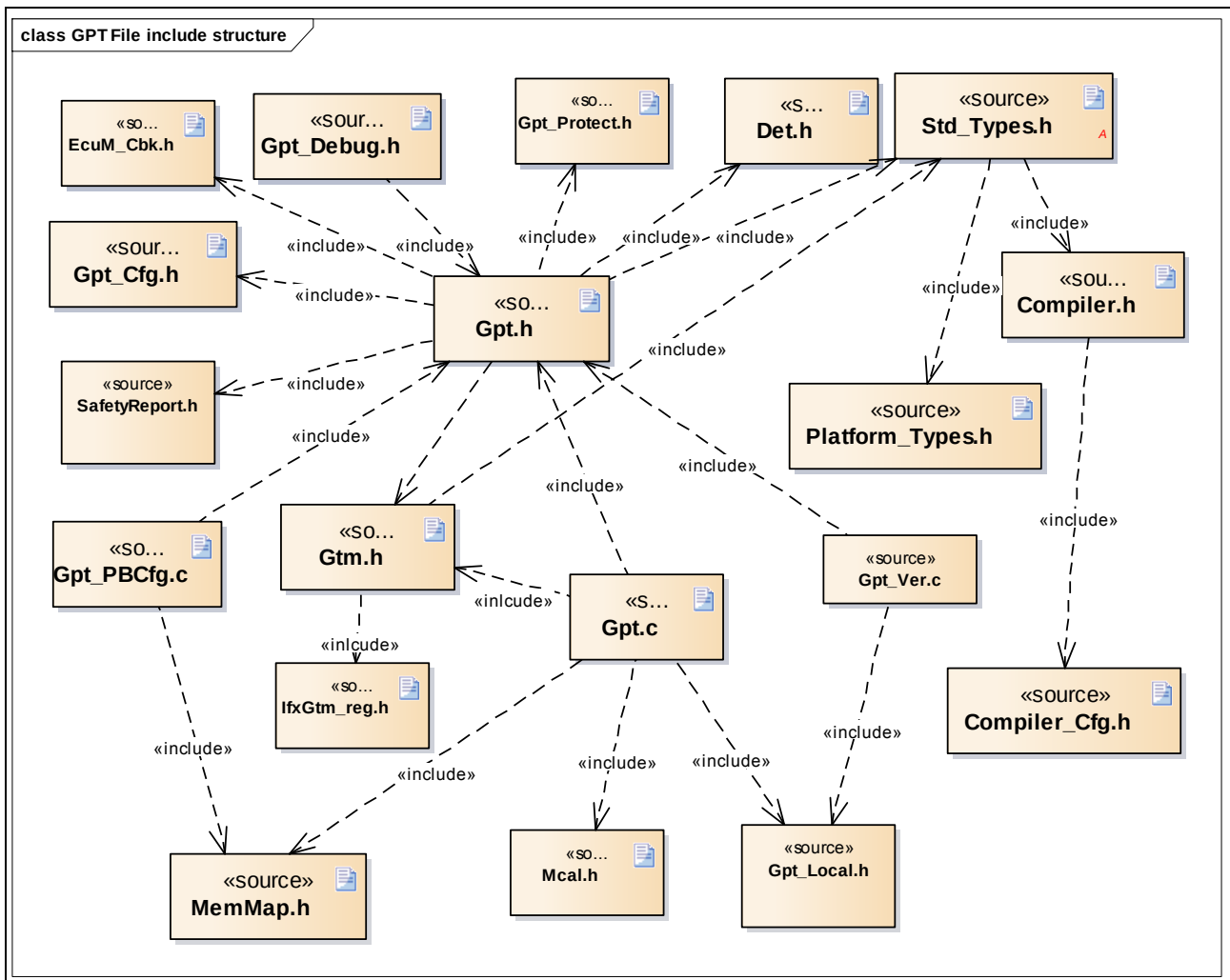


Figure 4 File Structure

The description of the contents of the file is listed in File Contents [Table 3](#).

Table 3 File Contents

File Name	File Contents
Platform_Types.h	Platform Dependencies (Tricore AURIX) are declared here.
Std_Types.h	Standard Data types to be used are declared here.
Compiler.h	Compiler Dependencies are abstracted here.
Compiler_Cfg.h	Contains abstraction of compiler specific keywords used for addressing data and code within declaration and definition.
MemMap.h	Mapping of code and data (variables, constant variables) to specific memory sections
IfxGtm_reg.h	Includes the register definition file for GTM.
Mcal.h	Provides library functions for MCAL layer.
Det.h	Provides the functionality of Development Error Tracer
Gtm.h	Contains all function definitions and macros of the GTM related section.
Gpt.h	Export of GPT Driver functionality.
Gpt_Local.h	This header file contains local macros, type definitions and function prototypes for the GPT Driver; which are not exposed to the upper layers.

Gpt.c	Implementation of GPT Driver functionality.
Gpt_Ver.c	This file contains AUTOSAR version specific functionality.
Gpt_Cfg.h	Configuration Data of the GPT Driver is declared here.
Gpt_PBCfg.c	Configuration Data of the GPT Driver is defined here.
EcuM_Cbk.h	Type definition for EcuM_WakeupSourceType and EcuM_CheckWakeup function.
Safety_Report.h	Header file for reporting safety related errors.
Gpt_Dbg.h	For exporting Debug related variables
Gpt_Protect.h	Included to define all the first level indirection macros for Protected mode support for supervisory access via OS calls

4 Configuration Documentation

The following chapters summarize all configuration parameters, their description and the type definition used to represent them in the code. This includes those specified in the AUTOSAR GPT Specification and the implementation specific parameters.

4.1 Configuration Concept

4.1.1 Configuration Class

The development of Basic Software Drivers involves the following development cycles:

- Compiling,
- Linking and
- Downloading of the executable to ECU memory.

Configuration of parameters can be done in any of these process-steps: pre-compile time, link time or even post-build time. According to the process-step that does the configuration of parameters, the configuration classes are categorized as below

Pre-compile time: The pre-compile time configuration is implemented by means of compiler switches or variables affecting the total build process. Pre-compiler configuration requires the Driver to be compiled, linked and located and hence the values of pre-compile parameters must be **frozen before compilation** and therefore can be used in source code distributions of the Driver only.

Link time: The Link time configuration doesn't affect compilation stage but impacts the linkage and locating stages. The Link time parameters can be used in binary (library) distributions of the Driver and the values of the Link time parameters are specified by means of external configuration source code that must be linked to the Driver during the build process.

Post-build time: The Post-build time configuration will not affect the build process and hence Post-build time configuration is applied to Driver at runtime. Since configuration location is statically independent of the Driver functionality itself but linked dynamically (e.g. at Driver initialization), post-build time configuration is used for re-programmable configuration data.

4.1.2 Configuration Variant

Based on the most suitable configuration class that may be needed for the configuration parameters of the Driver, a Driver can be delivered as the following variants

Variant PC: This variant is limited to pre-compile configuration parameters only. The intention of this variant is to optimize the parameters configuration for a source code delivery.

Variant LT: This variant allows a mix of pre-compile time- link time configuration parameters. The intention of this variant is to optimize the parameters configuration for an object code delivery.

Variant PB: This variant allows a mix of pre-compile time-, post build-time configuration parameters. The intention of this variant is to optimize the configuration parameters for a re-loadable binary.

The GPT Driver is delivered as a Variant PB.

4.1.3 How to read the Configuration Class field

The examples cited below help the user to identify the configuration class of the parameter for a given delivery type.

Topics [4.1.1](#), [4.1.2](#) should be read for understanding the configuration class field

4.2 Configuration Parameter Variance

This section describes the variance in the configuration parameter w.r.t supported microcontroller. GPT Driver doesn't have varying parameters across the supported microcontrollers

4.3 Container: GptDriverConfiguration

This section describes in detail the configuration provided by GPT Driver. This covers the configurations described by the AUTOSAR GPT Driver specification [\[1\]](#) and the microcontroller specific configuration.

Note: The “**configuration class**” details are given for all Variants specified in the SWS. The section “type” in the configuration table is explained only for Configuration Class “Post Build”. The configuration class states the possible support for each delivery.

4.3.1 GptDevErrorDetect

Table 4 GptDevErrorDetect

Name:	GptDevErrorDetect		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt_Cfg.h		
Range:	Values: true – STD_ON false – STD_OFF Default Value: STD_OFF		
Configuration Class:	Pre Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Switches the Development error Detection STD_ON or STD_OFF.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.2 GptReportWakeupSource

Table 5 GptReportWakeupSource

Name:	GptReportWakeupSource
Source:	This parameter is defined by the AUTOSAR, refer [1]
File:	Gpt_Cfg.h

Name:	GptReportWakeupSource		
Range:	Significant only if GptEnableWakeup for any one of the channels is TRUE and GptWakeupFunctionalityApi is STD_ON Values: STD_ON – Enabled STD_OFF – Disabled Default Value: STD_OFF		
Configuration Class:	Pre Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Enables/Disables wakeup source reporting. In Post Build configuration, if GptReportWakeupSource is set and one configuration has Wakeup capable channel then all the configurations should have atleast one wakeup capable channel.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	GptEnableWakeup, GptWakeupFunctionalityApi		

4.3.3 GptPBFixedAddress

Table 6 GptPBFixedAddress

Name:	GptPBFixedAddress		
Source:	This parameter is defined by Infineon		
File:	Gpt_Cfg.h		
Range:	Values: STD_ON – Fixed address feature used STD_OFF – Fixed address feature not used Default value: STD_OFF		
Configuration Class:	Pre Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	The fixed address feature supports the optimization of a post build configuration with a fixed address by avoiding unnecessary indirection.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.4 GptDebugSupport

Table 7 GptDebugSupport Configuration

Name:	GptDebugSupport
Source:	This parameter is defined by Infineon
File:	Gpt_Cfg.h
Range:	Values: ON – Enabled (true)

	OFF – Disabled (false) Default value: OFF		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Enables debugging support for the module. This shall provide the variables that are relevant for debugging purpose as Global variables.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.5 GptRunningInUser0Mode

Table 8 GptRunningInUser0Mode

Name:	GptRunningInUser0Mode		
Source:	This parameter is microcontroller specific		
File:	Gpt_Cfg.h		
Range:	Values: TRUE FALSE Default Value : FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If 'GptRunningInUser0Mode' is enabled then the API's selected by 'GptUserModeInitApiEnable' or 'GptUserModeDeInitApiEnable' or 'GptUserModeRuntimeApiEnable' will run in user mode 0. If the 'GptRunningInUser0Mode' switch is not selected and if any of the switches 'GptUserModeInitApiEnable', 'GptUserModeDeInitApiEnable', 'GptUserModeRuntimeApiEnable' are selected then those APIs will run in user mode 1. This switch selects between user mode0 and user mode 1 GptRunningInUser0Mode – STD_ON implies User Mode 0 GptRunningInUser0Mode – STD_OFF implies User Mode 1 if any of the switches (GptUserModeInitApiEnable or GptUserModeDeInitApiEnable or GptUserModeRuntimeApiEnable) are true else Supervisor Mode (If all the four switches mentioned are OFF).		
Scope:	Driver		
Dependency:	If 'GptRunningInUser0Mode' is enabled then at least any one of the switches 'GptUserModeInitApiEnable' or 'GptUserModeDeInitApiEnable' or 'GptUserModeRuntimeApiEnable' should be enabled.		

4.3.6 GptUserModeInitApiEnable

Table 9 GptUserModeInitApiEnable

Name:	GptUserModeInitApiEnable
--------------	--------------------------

Source:	This parameter is microcontroller specific		
File:	Gpt_Cfg.h		
Range:	Values: TRUE FALSE Default Value : FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If GptUserModeDeInitApiEnable is enabled then the Gpt_17_Gtm_Init API will run in User mode. Selection between User Mode 0 and User mode 1 is based on the switch GptRunningInUser0Mode		
Scope:	Driver		
Dependency:			

4.3.7 GptUserModeDeInitApiEnable

Table 10 GptUserModeDeInitApiEnable

Name:	GptUserModeDeInitApiEnable		
Source:	This parameter is microcontroller specific		
File:	Gpt_Cfg.h		
Range:	Values: TRUE FALSE Default Value : FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If GptUserModeDeInitApiEnable is enabled then the Gpt_17_Gtm_DeInit API will run in User mode. Selection between User Mode 0 and User mode 1 is based on the switch GptRunningInUser0Mode		
Scope:	Driver		
Dependency:			

4.3.8 GptUserModeRuntimeApiEnable

Table 11 GptUserModeRuntimeApiEnable

Name:	GptUserModeRuntimeApiEnable		
Source:	This parameter is microcontroller specific		
File:	Gpt_Cfg.h		
Range:	Values: TRUE FALSE Default Value : FALSE		

Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If GptUserModeRuntimeApiEnable is enabled then all APIs other than Gpt_17_Gtm_Init and Gpt_17_Gtm_DeInit APIs will run in User mode. Selection between User Mode 0 and User mode 1 is based on the switch GptRunningInUser0Mode		
Scope:	Driver		
Dependency:			

4.4 Container: GptConfigurationOfOptApiServices

4.4.1 GptVersionInfoApi

Table 12 GptVersionInfoApi

Name:	GptVersionInfoApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt_Cfg.h		
Range:	Values: true – STD_ON false – STD_OFF Default value: STD_OFF		
Configuration Class:	Pre Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	STD_ON Gpt_GetVersionInfo API is available to the user. STD_OFF Gpt_GetVersionInfo API is not available to the user.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None.		

4.4.2 GptDeInitApi

Table 13 GptDeInitApi

Name:	GptDeInitApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt_Cfg.h		
Range:	Values: true – STD_ON false – STD_OFF Default value: STD_OFF		
Configuration Class:	Pre Compile	X	All Variants
	Link Time	--	--

Name:	GptDelnitApi		
	Post Build	--	--
Description:	STD_ON Gpt_Delnit API is available to the user. STD_OFF Gpt_Delnit API is not available to the user.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.3 GptTimeElapsedApi

Table 14 GptTimeElapsedApi

Name:	GptTimeElapsedApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt_Cfg.h		
Range:	Values: true – STD_ON false – STD_OFF Default value: STD_OFF		
Configuration Class:	Pre Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	STD_ON Gpt_GetTimeElapsed API is available to the user. STD_OFF Gpt_GetTimeElapsed API is not available to the user.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.4 GptTimeRemainingApi

Table 15 GptTimeRemainingApi

Name:	GptTimeRemainingApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt_Cfg.h		
Range:	Values: true – STD_ON false – STD_OFF Default value: STD_OFF		
Configuration Class:	Pre Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	STD_ON Gpt_GetTimeRemaining API is available to the user. STD_OFF Gpt_GetTimeRemaining API is not available to the user.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.5 GptEnableDisableNotificationApi

Table 16 GptEnableDisableNotificationApi

Name:	GptEnableDisableNotificationApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt_Cfg.h		
Range:	Significant only if GptNotification for any one of the channels is not NULL Values: true – STD_ON false – STD_OFF Default value: STD_OFF		
Configuration Class:	Pre Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	STD_ON Gpt_EnableNotification and Gpt_DisableNotification API's are available to the user. STD_OFF Gpt_EnableNotification and Gpt_DisableNotification API's are not available to the user.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	GptNotification		

4.4.6 GptWakeupFunctionalityApi

Table 17 GptWakeupFunctionalityApi

Name:	GptWakeupFunctionalityApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt_Cfg.h		
Range:	Significant only if GptEnableWakeup for any one of the channels is TRUE Values: true – STD_ON false – STD_OFF Default value: STD_OFF		
Configuration Class:	Pre Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	STD_ON Gpt_SetMode, Gpt_CheckWakeup, Gpt_EnableWakeup and Gpt_DisableWakeup API's are available to the user. STD_OFF Gpt_SetMode, Gpt_EnableWakeup and Gpt_DisableWakeup API's are not available to the user.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	GptEnableWakeup		

4.5 Container: GptChannelConfigSet

4.5.1 Container: GptChannelConfiguration

This container contains the channel-wide configuration (parameters) of the GPT Driver.

4.5.1.1 GptChannelId

Table 18 GptChannelId

Name:	GptChannelId		
Source:	This parameter is defined by the AUTOSAR		
File:	Gpt_PBCfg.c		
Range:	Values: Unique number in the Container Default value: Current Channel Index		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Channel Id of the GPT channel. This value will be assigned to the symbolic name derived of the GptChannelConfiguration container short name. If the Symbolic name assigned to the channel is "OS_TRIGGER", then the symbolic name will be available in the format "GPT_OS_TRIGGER"		
SFRs Modified	None		
Scope:	Driver		
Dependency:			

4.5.1.2 GptEnableWakeup

Table 19 GptEnableWakeup

Name:	GptEnableWakeup		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt_PBCfg.c		
Range:	Values: TRUE - Wakeup capability on this channel is enabled FALSE - Wakeup capability on this channel is disabled Default value: FALSE		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Enables wakeup capability for a channel when timeout period expires.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	Parameter falls into configuration structure only when		

Name:	GptEnableWakeup
	GPT_WAKEUP_FUNCTIONALITY_API is enabled.

4.5.1.3 GptChannelMode

Table 20 GptChannelMode

Name:	GptChannelMode		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt_PBCfg.c		
Range:	Values: GPT_CH_MODE_ONESHOT - Timer channel stops after reaching its end value. GPT_CH_MODE_CONTINUOUS - Timer channel is restarted automatically after reaching its end value. Default value: GPT_CH_MODE_CONTINUOUS		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Specifies the behavior of the timer channel after the timeout has occurred.		
SFRs Modified	TOMi_CHx_CTRL/ATOMi_CHx_CTRL(for i and x refer Table 1)		
Scope:	Driver		
Dependency:	In case of GPT Channel is One Shot Mode, configure the priority for the respective TOM/ATOM channel SRN node in IRQ module, the configuration is mandatory otherwise the channel will not work properly.		

4.5.1.4 GptSignalType

Table 21 GptSignalType

Name:	GptSignalType		
Source:	This parameter is added for Safety purposes		
File:	Gpt_PBCfg.c		
Range:	Values: GPT_QM_SIGNAL - Timer channel is QM type. GPT_ASIL_SIGNAL - Timer channel is ASIL type Default value: GPT_QM_SIGNAL		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Specifies the timer channel signal type.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	The parameter is available in configuration structure only when GptSafetyEnable is enabled.		

4.5.1.5 GptChannelTickFrequency

Table 22 GptChannelTickFrequency

Name:	GptChannelTickFrequency		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	-		
Range:	Values: -		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Specifies the tick frequency of the timer channel in Hz. This is not supported as Clock is handled in GTM		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.1.6 GptChannelTickValueMax

Table 23 GptChannelTickValueMax

Name:	GptChannelTickValueMax		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	-		
Range:	Values: -		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Specifies the maximum value of ticks, the timer is able to count. This is not supported as Clock is handled in GTM		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.1.7 GptNotification

Table 24 GptNotification

Name:	GptNotification		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt_PBCfg.c		
Range:	Values: Gpt notification pointer of type FUNCTION_NAME(Selectable) / Address (Loadable) Default value: NULL_PTR		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--

	Post Build	X	Variant PB
Description:	Notification Pointer function or callback function: When the timeout happens this function will be called by the GPT Driver for each individual channel. If notification is not desired, then this parameter should be removed i.e. Multiplicity of 0..1		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.1.8 GptAssignedHwUnit

This parameter contains the configuration needed to configure the hardware resources for a timer channel. To configure a timer channel the required hardware resources are GTM Hardware TOM/ATOM channels.

This container is Implementation specific defined for GPT Driver only.

Table 25 GptAssignedHwUnit

Name:	GptAssignedHwUnit		
Source:	This parameter is microcontroller specific		
File:	Gpt_PBCfg.c		
Range:	Values: TOM or ATOM channel Number.		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	The hardware resource needed to implement the GPT timer is chosen using this parameter. For notification to be received, the corresponding SRN need to be assigned a priority in IRQ.		
SFRs Modified	None		
Scope	Driver		
Dependency:	None		

4.5.1.9 GptClockReferencePoint

This container contains a parameter, which represents a reference to a container of the type McuClockReferencePoint (defined in module MCU). A container is needed to support multiple clock references (hardware dependent).

4.5.1.9.1 GptClockReference

Table 26 GptClockReference

Name:	GptClockReference		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Gpt.xdm		
Range:	Reference to McuClockReferencePoint		
Configuration Class:	Pre Compile	X	All Variants

	Link Time	--	--
	Post Build	--	--
Description:	Reference to a container of the type McuClockReferencePoint, to select an input clock. The configuration editor for the GPT module can support the integrator by only allowing a selection of those clock reference points that can be connected physically to the GPT hardware peripheral. The desired frequency (desired by GPT) has to be the same as the selected and provided frequency of the MCU configuration. This has to be checked automatically. This Parameter is not supported as Clock configuration is handled in GTM.		
SFRs Modified	None		
Scope	Driver		
Dependency:	None		

4.5.1.10 Container: GptWakeupConfiguration

This container contains the configuration needed to configure a wakeup capable channel

4.5.1.11 GptWakeupSourceRef

Table 27 GptWakeupSourceRef

Name:	GptWakeupSourceRef		
Source:	This parameter is defined by the AUTOSAR		
File:	Gpt_PBCfg.c		
Range:	Significant only if GptEnableWakeup is TRUE Values: This is a reference parameter of type EcuM_WakeupSourceType		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	In case the wakeup-capability is true the value configured for the reference in EcuM is transmitted to the ECU State Manager		
SFRs Modified	None		
Scope:	ECU		
Dependency:	The parameter is available in configuration structure only when GptWakeupFunctionalityApi and GptReportWakeupSource both are enabled.		

4.5.1.12 GptChannelClkSrcRef

Table 28 GptChannelClkSrcRef

Name:	GptChannelClkSrcRef
Source:	This parameter is defined by the AUTOSAR
File:	Gpt.xdm

CONFIDENTIAL

Range:	Values: -		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Reference to the GptClockReferencePoint from which the channel clock is derived. This is not supported as Clock is handled in GTM		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.6 Container: CommonPublishedInformation

It contains published information about vendor and versions.

4.6.1 ArMajorVersion

Table 29 ArMajorVersion

Name:	ArMajorVersion		
Source:	This is Infineon specific parameter.		
File:	Gpt.xdm/ Gpt.bmd		
Range:	-		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Major version number of AUTOSAR specification on which the appropriate implementation is based on.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.6.2 ArMinorVersion

Table 30 ArMinorVersion

Name:	ArMinorVersion		
Source:	This is Infineon specific parameter.		
File:	Gpt.xdm/ Gpt.bmd		
Range:	-		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.		
SFRs Modified	None		

CONFIDENTIAL

Scope:	Driver
Dependency:	None

4.6.3 ArPatchVersion

Table 31 ArPatchVersion

Name:	ArPatchVersion		
Source:	This is Infineon specific parameter.		
File:	Gpt.xdm/Gpt.bmd		
Range:	-		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Patch level version number of AUTOSAR specification on which the appropriate implementation is based on.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.6.4 SwMajorVersion

Table 32 SwMajorVersion

Name:	SwMajorVersion		
Source:	This is Infineon specific parameter.		
File:	Gpt.xdm/Gpt.bmd		
Range:	-		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Major version number of the vendor specific implementation of the module.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.6.5 SwMinorVersion

Table 33 SwMinorVersion

Name:	SwMinorVersion		
Source:	This is Infineon specific parameter.		
File:	Gpt.xdm/Gpt.bmd		
Range:	-		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--

CONFIDENTIAL

Description:	Minor version number of the vendor specific implementation of the module.
SFRs Modified	None
Scope:	Driver
Dependency:	None

4.6.6 SwPatchVersion

Table 34 SwPatchVersion

Name:	SwMajorVersion		
Source:	This is Infineon specific parameter.		
File:	Gpt.xdm/Gpt.bmd		
Range:	-		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Patch level version number of the vendor specific implementation of the module.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.6.7 ModuleId

Table 35 ModuleId

Name:	ModuleId		
Source:	This is Infineon specific parameter.		
File:	Gpt.xdm/Gpt.bmd		
Range:	100		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Module ID of GPT module from Module List		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.6.8 VendorId

Table 36 VendorId

Name:	VendorId		
Source:	This is Infineon specific parameter.		
File:	Gpt.xdm/Gpt.bmd		
Range:	17		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--

CONFIDENTIAL

	Post Build	--	--
Description:	Vendor ID of the GPT module according to the AUTOSAR vendor list.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.6.9 VendorApilnfix

Table 37 VendorApilnfix

Name:	VendorApilnfix		
Source:	This is Infineon specific parameter.		
File:	Gpt.xdm/Gpt.bmd		
Range:	None		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	In driver modules which can be instantiated several times on a single ECU then the name of APIs is extended by the VendorId and a vendor specific name. This parameter is used to specify the vendor specific name.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.6.10 Release

Table 38 Release

Name:	Release
Source:	This is Infineon specific parameter.
File:	Gpt.xdm/Gpt.bmd
Range:	_TRICORE_TC212 _TRICORE_TC213 _TRICORE_TC214 _TRICORE_TC222 _TRICORE_TC223 _TRICORE_TC224 _TRICORE_TC275 _TRICORE_TC277 _TRICORE_TC264 _TRICORE_TC265 _TRICORE_TC297 _TRICORE_TC298

CONFIDENTIAL

	_TRICORE_TC299		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Aurix derivative used for the implementation.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.7 Container: GptSafety

This container contains the configuration parameters for the safety features of the GPT driver.

4.7.1 GptSafetyEnable

Table 39 GptSafetyEnable

Syntax:	GptSafetyEnable		
Source:	This parameter is defined by IFX		
File:	Gpt_Cfg.h		
Range:	Values: TRUE - Safety features enabled for GPT driver FALSE – Safety features disabled Default value: FALSE		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the safety features of GPT driver. By enabling this flag, implicitly the following safety features are enabled: - Range check on API parameters is done by GPT Driver. - GPT configuration structure will have a unique marker field which will be verified by the driver - Gpt_SetMode will check to ensure safe setting of Mode by ensuring that for any running GPT channel configured for WakeUp with wakeup enabled, the IRQ bits for the channel are enabled before going to SLEEP - Publish the safety usage of GPT driver for user to set the respective MPU settings <pre>#define GPT_SAFETY_ENABLE (STD_OFF/STD_ON)</pre>		
SFRs Modified	None		
Scope:	Driver.		
Dependency:	None		

4.7.2 GptInitCheckApi

Table 40 GptInitCheckApi

Syntax:	GptInitCheckApi		
Source:	This parameter is defined by IFX		
File:	Gpt_Cfg.h		
Range:	Values: TRUE - Gpt_InitCheck() API will be enabled FALSE - Gpt_InitCheck() API will be disabled Default value: TRUE		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the safety feature Gpt_InitCheck() which verifies the initialization done by GPT driver. #define GPT_INITCHECK_API (STD_OFF/ON) This #define will not be generated if GptSafetyEnable is FALSE.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	GptSafetyEnable		

4.7.3 GptGetModeApi

Table 41 GptGetModeApi

Syntax:	GptGetModeApi		
Source:	This parameter is defined by IFX		
File:	Gpt_Cfg.h		
Range:	Values: TRUE - Gpt_GetMode() API will be enabled FALSE - Gpt_GetMode() API will be disabled Default value: TRUE		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the safety feature Gpt_GetMode() which returns the driver mode of GPT. #define GPT_GETMODE_API (STD_OFF/ON) This #define will not be generated if GptSafetyEnable is FALSE.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	GptSafetyEnable		

5 Published parameters

This section describes the parameters published by the GPT Driver.

5.1 VendorId

Table 42 VendorId

Syntax:	GPT_VENDOR_ID
Type:	Macro - #define
File:	Gpt.h
Value:	17
Description:	Vendor is Infineon Technologies

5.2 ModuleId

Table 43 ModuleId

Syntax:	GPT_MODULE_ID
Type:	Macro - #define, type casted to uint16
File:	Gpt.h
Value:	100
Description:	This macro gives the GPT Driver module ID as described by AUTOSAR

5.3 InstanceId

Table 44 InstanceId

Syntax:	GPT_INSTANCE_ID
Type:	Macro - #define, type casted to uint16
File:	Gpt_Cfg.h
Value:	0
Description:	This macro gives the Gpt driver Instance ID as described by AUTOSAR

5.4 ARMajorVersion

Table 45 ARMajorVersion

Syntax:	GPT_AR_RELEASE_MAJOR_VERSION
Type:	Macro - #define, type casted to uint16
File:	Gpt_Cfg.h
Value:	Depends on the delivery
Description:	Major version number of the AUTOSAR specification.

5.5 ARMinorVersion

Table 46 ARMinorVersion

Syntax:	GPT_AR_RELEASE_MINOR_VERSION
Type:	Macro - #define, type casted to uint16
File:	Gpt_Cfg.h
Value:	Depends on the delivery
Description:	Minor version number of the AUTOSAR specification.

5.6 ARPatchVersion

Table 47 ARPatchVersion

Syntax:	GPT_AR_RELEASE_REVISION_VERSION
Type:	Macro - #define, type casted to uint16
File:	Gpt_Cfg.h
Value:	Depends on the delivery
Description:	Patch version number of the AUTOSAR specification.

5.7 SwMajorVersion

Table 48 SwMajorVersion

Syntax:	GPT_SW_MAJOR_VERSION
Type:	Macro – #define
File:	Gpt_Cfg.h
Value:	Depends on the delivery
Description:	Major version number of the vendor specific implementation of the Driver. The numbering is vendor specific. The MAJOR_VERSION is incremented if the Driver is not downwards compatible any more (e.g. existing API changed)

5.8 SwMinorVersion

Table 49 SwMinorVersion

Syntax:	GPT_SW_MINOR_VERSION
Type:	Macro - #define
File:	Gpt_Cfg.h
Value:	Depends on the delivery
Description:	Minor version number of the vendor specific implementation of the Driver. The numbering is vendor specific. MINOR_VERSION is incremented if the Driver is still downwards compatible (e.g. new functionality added)

5.9 SwPatchVersion

Table 50 SwPatchVersion

Syntax:	GPT_SW_PATCH_VERSION
Type:	Macro - #define
File:	Gpt_Cfg.h
Value:	Depends on the delivery
Description:	Patch level version number of the vendor specific implementation of the Driver. The numbering is vendor specific. The PATCH_VERSION is incremented if the Driver is still upwards and downwards compatible (e.g. bug fixed)

6 API Documentation

The API documentation describes the syntax and semantics of the API functions as well as it gives some additional implementation specific behavior description not defined by the AUTOSAR GPT Driver specification.

Implementation specific type definitions, which are required for configuration and convenience, are also documented.

6.1 API Type Definitions

6.1.1 Type Definition Gpt_ConfigType

Table 51 Type Definition Gpt_ConfigType

Syntax:	Gpt_ConfigType	
Type:	Struct	
File:	Gpt.h	
Range:	ChannelConfigPtr (Gpt_ChannelConfigType*)	Pointer to Gpt_ChannelConfigType (Refer Table 52)
	Gpt_MaxChannels (Gpt_ChannelType)	Number of GPT Channels present in configuration.
Description:	This type definition is used to configure overall GPT configuration. The pointer to the object of this type is used in Gpt_Init API to initialize the GPT Driver.	

6.1.2 Type Definition Gpt_ChannelConfigType

Table 52 Type Definition Gpt_ChannelConfigType

Syntax:	Gpt_ChannelConfigType	
Type:	struct	
File:	Gpt.h	
Range:	Gpt_NotifFunctionPtrType (Function Pointer)	Gpt_Notification - Function pointer for normal mode notification
	EcuM_WakeupSourceType	Gpt_ChannelWakeupInfo - Wakeup information to EcuM_CheckWakeup
	Gtm_GlobalChannelNumberType (uint16)	Gpt_AssignedHwUnit - Hardware resources (TOM/ATOM Channel) used for the channel.
	Gpt_EnableWakeupType	Gpt_EnableWakeupState - Channel wakeup capability TRUE/FALSE
	Gpt_ModeType	Gpt_ChannelMode - Channel mode - GPT_MODE_CONTINUOUS/ GPT_MODE_ONESHOT

	Gpt_SignalType	Gpt_SignalType- GPT_QM_SIGNAL/ GPT_ASIL_SIGNAL. Required only when GPT_SAFETY_ENABLE is STD_ON
	Gpt_ClockType	Clock divider selection for TOM/ATOM channels
Description:	This is the type of the data structure containing Timer Channel Specific Parameters. A GPT configuration is made of one or more timer channels.	

6.1.3 Type Definition Gpt_ChannelType

Table 53 Gpt_ChannelType

Syntax:	Gpt_ChannelType
Type:	Uint8
File:	Gpt.h
Range:	0 to 31(Maximum number of channels configurable)
Description:	Numeric ID of a GPT channel.

6.1.4 Type Definition Gpt_ValueType

Table 54 Gpt_ValueType

Syntax:	Gpt_ValueType
Type:	uint32
File:	Gpt.h
Range:	0 to 0xFFFFFFFF
Description:	Used for reading the current timer value/setting periodic timer values (in number of ticks).

6.1.5 Type Definition Gpt_ModeType

Table 55 Type Definition Gpt_ModeType

Syntax:	Gpt_ModeType	
Type:	uint8	
File:	Gpt.h	
Range:	GPT_MODE_UNINITIALIZED	Uninitialized mode
	GPT_MODE_NORMAL	normal mode
	GPT_MODE_SLEEP	sleep mode
Description:	Allows the selection of different power modes GPT_MODE_UNINITIALIZED, GPT_MODE_NORMAL and GPT_MODE_SLEEP.	

6.1.6 Type Definition Gpt_EnableWakeupType

Table 56 Type Definition Gpt_EnableWakeupType

Syntax:	Gpt_EnableWakeupType
Type:	Boolean
File:	Gpt.h
Range:	TRUE – channel is wakeup enabled. FALSE – channel is wakeup disabled.
Description:	This type is used to Enable/Disable the channel wakeup status.

6.1.7 Type Definition EcuM_WakeupSourceType

Table 57 Type Definition EcuM_WakeupSourceType

Syntax:	EcuM_WakeupSourceType
Type:	uint32
File:	EcuM_Cbk.h
Range:	As defined by EcuM configuration.
Description:	Type definition for Wakeup Source Type. (Imported from EcuM.h)

6.1.8 Type Definition Gpt_ClockType

Table 58 Gpt_ClockType

Syntax:	Gpt_ClockType		
Type:	uint8		
File:	Gpt.h		
Range:	TOM	GTM_FIXED_CLOCK_0	0
		GTM_FIXED_CLOCK_1	1
		GTM_FIXED_CLOCK_2	2
		GTM_FIXED_CLOCK_3	3
		GTM_FIXED_CLOCK_4	4
	ATOM	GTM_CONFIGURABLE_CLOCK_0	0
		GTM_CONFIGURABLE_CLOCK_1	1
		GTM_CONFIGURABLE_CLOCK_2	2
		GTM_CONFIGURABLE_CLOCK_3	3
		GTM_CONFIGURABLE_CLOCK_4	4
		GTM_CONFIGURABLE_CLOCK_5	5
		GTM_CONFIGURABLE_CLOCK_6	6
		GTM_CONFIGURABLE_CLOCK_7	7
Description:	Used for selection of various Clock divider values for TOM/ATOM channels		

6.1.9 Type Definition Gpt_SignalType

Table 59 Gpt_SignalType

Syntax:	Gpt_SignalType	
Type:	uint8	
File:	Gpt.h	
Range:	GPT_QM_SIGNAL	0
	GPT_ASIL_SIGNAL	1
Description:	Used for selection of the channel to be ASIL or QM signal.	

6.1.10 Type Definition Gpt_ChannelIdentifierType

Table 60 Gpt_ChannelIdentifierType

Syntax:	Gpt_ChannelIdentifierType	
Type:	Struct	
File:	Gpt.h	
Range:	uint8	ModuleId- To determine channel to be in TOM or ATOM.
	uint8	ModuleNo- To determine x or y in TOMx or ATOMy
	uint8	ChannelNo- To determine the channel number in TOMx or ATOMy
	uint8	GroupNo- To determine which TGC the channel belongs in TOMx
Description:	Used to determine which TOM/ATOM channel the GPT channel belongs	

6.2 API Functions

This chapter describes the API interfaces provided by GPT Driver.

6.2.1 Gpt_GetVersionInfo

Table 61 Specification of API Gpt_GetVersionInfo

Service Name:	Gpt_GetVersionInfo
Syntax:	void Gpt_GetVersionInfo (Std_VersionInfoType *VersionInfoPtr)
Source	This API is defined by the AUTOSAR.
Service ID:	0x00
Sync / Async:	Synchronous
Re-entrancy:	Non reentrant

Parameters (in):	None
Parameters (out):	VersionInfoPtr: Pointer to store the Driver Version Information.
Return Value:	None
Description:	Service to return the version information of the Driver. This is implemented as a Macro
Caveats:	None
Configuration:	GPT_VERSION_INFO_API
DET:	GPT_E_PARAM_POINTER is reported when API called with VersionInfoPtr as NULL_PTR
DEM:	None
Implementation Comments:	
I/O Mode	This API can be invoked when the CPU is in either User-1 Mode or User-0 Mode or Supervisor Mode.

6.2.2 Gpt_Init

Table 62 Specification of API Gpt_Init

Service Name:	Gpt_Init	
Syntax:	void Gpt_Init (const Gpt_ConfigType *ConfigPtr)	
Source	This API is defined by the AUTOSAR.	
Service ID:	0x01	
Sync / Async:	Synchronous	
Re-entrancy:	Non Reentrant	
Parameters (in):	ConfigPtr	Pointer to the configuration set.
Parameters (out):	None	
Return Value:	None	
Description:	Service for initializing the GPT Driver according to the given configuration.	
Caveats:	This Service cannot be called again before calling Gpt_DeInit. Gpt_Init shall be invoked by the user only after PLL setup (Using Mcu_DistributePllClock).	
Configuration:	None	
DET:	GPT_E_PARAM_POINTER : NULL pointer is passed as input parameter GPT_E_ALREADY_INITIALIZED: Driver is already initialized. In case of GptPBFixedAddress selected the user must call Gpt_Init with post build configuration instance 0 otherwise DET GPT_E_PARAM_POINTER will be reported.	
DEM:	None	
Safety Error reporting:	GPT_E_PARAM_POINTER : If parameter ConfigPtr is a null pointer or if Marker of the ConfigPtr does not match	
Implementation Comments:	- Mcu_Init should be executed before calling this service. Mcu_Init will initialize GTM resources. - Mcu_Init performs GTM SFR reset if McuResetSfrAtInit is enabled.	

I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is 'True' and GptUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeInitApiEnable is 'False'.
-----------------	---

6.2.3 Gpt_DeInit

Table 63 Specification of API Gpt_DeInit

Service Name:	Gpt_DeInit
Syntax:	void Gpt_DeInit (void)
Source	This API is defined by the AUTOSAR.
Service ID:	0x02
Sync / Async:	Synchronous
Re-entrancy:	Non re-entrant
Parameters (in):	None
Parameters (out):	None
Return Value:	None
Description:	Service for deinitializing all hardware timer channels to their power on reset state. This service also disables notifications and wakeup events.
Caveats:	None
Configuration:	This function is pre-compile time configurable STD_ON / STD_OFF by the configuration parameter GptDeinitApi.
DET:	GPT_E_UNINIT : Driver not initialized GPT_E_BUSY : At least one of the channel is running
DEM:	-
Implementation Comments:	None
I/O Mode	GptUserModeDeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeDeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeDeInitApiEnable is 'False'.

6.2.4 Gpt_GetTimeElapsed

Table 64 Specification of API Gpt_GetTimeElapsed

Service Name:	Gpt_GetTimeElapsed
----------------------	--------------------

Syntax:	Gpt_ValueType Gpt_GetTimeElapsed (Gpt_ChannelType Channel)			
Source	This API is defined by the AUTOSAR.			
Service ID:	0x03			
Sync / Async:	Synchronous			
Re-entrancy:	Reentrant			
Parameters (in):	Channel	The associated Channel where the time elapsed needs to be calculated		
Parameters (out):	None			
Return Value:	Gpt_ValueType			
Description:	Service for querying the time already elapsed in a channel.			
	State/ Circumstance	Timer Channel State	Return Value	Development Error (If Enabled)
	Driver Uninitialized	-	0	GPT_E_UNINIT
	Driver Initialized	Initialized	0	-
		Running	Elapsed time	-
		Stopped	Elapsed time at the moment of Stopping	-
		Expired (in case of One Shot Mode)	Target time	-
	Invalid Parameter “Channel”	all	0	GPT_E_PARAM_CHANNEL
Caveats:	Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel.			
Configuration:	This function is pre-compile time configurable STD_ON / STD_OFF by the configuration parameter GptTimeElapsedApi.			
DET:	GPT_E_UNINIT		Driver not initialized	
	GPT_E_PARAM_CHANNEL		Invalid Channel Id (Greater then number of channels configured)	
DEM:	None			
Safety Error reporting:	GPT_E_PARAM_CHANNEL: If the parameter Channel is invalid			
Implementation Comments:	None			

I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is 'True' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'False'.
-----------------	--

6.2.5 Gpt_GetTimeRemaining

Table 65 Specification of API Gpt_GetTimeRemaining

Service Name:	Gpt_GetTimeRemaining	
Syntax:	Gpt_ValueType Gpt_GetTimeRemaining	
	(	
	Gpt_ChannelType Channel	
	)	
Source	This API is defined by the AUTOSAR.	
Service ID:	0x04	
Sync / Async:	Synchronous	
Re-entrancy:	Re-entrant	
Parameters (in):	Channel	The associated Channel where the time remaining needs to be calculated
Parameters (out):	None	
Return Value:	Gpt_ValueType	

Description:	This function returns the timer value remaining until the next timeout period expires (calculated by subtracting the current timer value from the timeout value and returning the absolute value).			
	State/ Circumstance	Timer Channel State	Return Value	Development Error (If Enabled)
	Driver Uninitialized	-	0	GPT_E_UNINIT
	Driver Initialized	Initialized	0	-
		Running	Remaining time	-
		Stopped	Remaining time at moment of Stopping	-
		Expired (Only One Shot Mode)	0	-
Invalid Parameter “Channel”	All	0	GPT_E_PARAM_CHANNEL	
Caveats:	Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel.			
Configuration:	This function is pre-compile time configurable STD_ON / STD_OFF by the configuration parameter GptTimeRemainingApi.			
DET:	GPT_E_UNINIT		Driver not initialized	
	GPT_E_PARAM_CHANNEL		Invalid Channel Id (Greater then number of channels configured)	
DEM:	None			
Safety Error reporting:	GPT_E_PARAM_CHANNEL: If the parameter Channel is invalid			
Implementation Comments:	None			
I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is ‘True’ and GptUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is ‘False’ and GptUserModeRuntimeApiEnable is ‘True’. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is ‘False’ and GptUserModeRuntimeApiEnable is ‘False’.			

6.2.6 Gpt_StartTimer

Table 66 Specification of API Gpt_StartTimer

Service Name:	Gpt_StartTimer	
Syntax:	<pre>void Gpt_StartTimer (Gpt_ChannelType Channel Gpt_ValueType Value)</pre>	
Source	This API is defined by the AUTOSAR.	
Service ID:	0x05	
Sync / Async:	Synchronous	
Re-entrancy:	Re-entrant	
Parameters (in):	Channel	The associated Channel where the timer needs to be started
	Value	Timeout value
Parameters (out):	None	
Return Value:	None	
Description:	Service for starting the selected timer channel with a defined timeout period. After this timeout period, a notification can be invoked (if enabled). Timeout value has to be entered in clock ticks.	
Caveats:	Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel.	
Configuration:	None	
DET:	GPT_E_UNINIT	Driver not initialized
	GPT_E_PARAM_VALUE	1) Invalid Value or Value out of range. In TOM: If (Value > 0xFFFF) In ATOM: If (Value > 0xFFFFFFF) 2) Invalid Value if "Value" is Zero
	GPT_E_PARAM_CHANNEL	Invalid Channel Id (Greater then number of channels configured)
	GPT_E_BUSY	Channel already started
DEM:	None	
Safety Error reporting:	GPT_E_PARAM_VALUE	1) Invalid Value or Value out of range. In TOM: If (Value > 0xFFFF) In ATOM: If (Value > 0xFFFFFFF) 2) Invalid Value if "Value" is Zero
	GPT_E_PARAM_CHANNEL	Invalid Channel Id (greater than number of channels configured)

Implementation Comments:	- Starts the Timer channel specified with the timeout value. The maximum timeout value differs for the different timer types. In case of TOM channel, the maximum Timeout possible is 0xFFFF. In case of ATOM channel, the maximum Timeout possible is 0xFFFFF. - Timeout value in terms of time can be calculated considering the Clock configured for the TOM/ATOM channel in GTM. Hence depending on the requirement, the corresponding Fixed or Configurable clock need to be configured in GTM.
I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is 'True' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'False'.

6.2.7 Gpt_StopTimer

Table 67 Specification of API Gpt_StopTimer

Service Name:	Gpt_StopTimer	
Syntax:	<pre>void Gpt_StopTimer (Gpt_ChannelType Channel)</pre>	
Source	This API is defined by the AUTOSAR.	
Service ID:	0x06	
Sync / Async:	Synchronous	
Re-entrancy:	Re-entrant	
Parameters (in):	Channel	The associated Channel where the timer needs to be stopped
Parameters (out):	None	
Return Value:	None	
Description:	Service for stopping the selected timer channel.	
Caveats:	Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel.	
Configuration:	None	
DET:	GPT_E_UNINIT : Driver not initialized	
	GPT_E_PARAM_CHANNEL : Invalid Channel Id (Greater then number of channels configured)	
DEM:	None	
Safety Error reporting:	GPT_E_PARAM_CHANNEL	If the Channel is not within range
Implementation Comments:	None	

I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is 'True' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'False'.
-----------------	--

6.2.8 Gpt_EnableNotification

Table 68 Specification of API Gpt_EnableNotification

Service Name:	Gpt_EnableNotification	
Syntax:	<pre>void Gpt_EnableNotification (Gpt_ChannelType Channel)</pre>	
Source	This API is defined by the AUTOSAR.	
Service ID:	0x07	
Sync / Async:	Synchronous	
Re-entrancy:	Reentrant	
Parameters (in):	Channel	The associated Channel where the timer needs to be stopped
Parameters (out):	None	
Return Value:	None	
Description:	Service for enabling the notification for a channel during runtime. This function can be called, while the timer is already running.	
Caveats:	Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel.	
Configuration:	This function is pre-compile time configurable STD_ON / STD_OFF by the configuration parameter GptEnableDisableNotificationApi.	
DET:	GPT_E_UNINIT : Driver not initialized GPT_E_PARAM_CHANNEL : Invalid Channel Id (Greater then number of channels configured) or if no Notification is configured for the channel.	
DEM:	None	
Safety Error reporting:	GPT_E_PARAM_CHANNEL	If the Channel is not within range
Implementation Comments:	For notifications to be available, the SRN for the TOM/ATOM channel chosen in the GPT channel, has to be provided a valid SRPN number in the IRQ/OS module. In case of TOM/ATOM, 2 successive channel share the same SRN. This is needed to be provided a valid Service Request priority number.	

I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is 'True' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'False'.
-----------------	--

6.2.9 Gpt_DisableNotification

Table 69 Functional Specification - Gpt_DisableNotification

Service name:	Gpt_DisableNotification	
Syntax:	<pre>void Gpt_DisableNotification (Gpt_ChannelType Channel)</pre>	
Source	This API is defined by the AUTOSAR.	
Service ID:	0x08	
Sync/Async:	Synchronous	
Reentrancy:	Reentrant	
Parameters (in):	Channel	The associated Channel where the timer needs to be stopped
Parameters (out):	None	
Return value:	None	
Description:	Service for disabling the notification for a channel during runtime. This function can be called, while the timer is already running.	
Caveats:	Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel.	
Configuration:	This function is pre-compile time configurable STD_ON / STD_OFF by the configuration parameter GptEnableDisableNotificationApi.	
DET:	GPT_E_UNINIT	Gpt_DisableNotification function called prior to init function
	GPT_E_PARAM_CHANNEL	Invalid Channel Id (Greater then number of channels configured) or if no valid notification is configured for the channel.
DEM:	None	
Safety Error reporting:	GPT_E_PARAM_CHANNEL	If the Channel is not within range
Implementation Comments:	None	
I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is 'True' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided -	

	configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'False'.
--	--

6.2.10 Gpt_SetMode

Table 70 Functional Specification - Gpt_SetMode

Service name:	Gpt_SetMode	
Syntax:	<pre>void Gpt_SetMode (Gpt_ModeType mode)</pre>	
Source	This API is defined by the AUTOSAR.	
Service ID:	0x09	
Sync/Async:	Synchronous	
Reentrancy:	Non-reentrant	
Parameters (in):	GPT_MODE_NORMAL	All notifications are available according to notification requests.
	GPT_MODE_SLEEP	In sleep mode notifications are not available.
Parameters (out):	None	
Return value:	None	
Description:	GPT_MODE_NORMAL mode all notifications are available according to notification requests. GPT_MODE_SLEEP Only wakeup capable channels with wakeup enabled will remain on and all other channels (channels that were wakeup disabled (using Gpt_DisableWakeup) or non wakeup capable channels) are stopped. Note: - The timer channels which have been stopped by entering the sleep mode are not restarted automatically on choosing Normal Mode. - The notifications and wakeup capabilities will remain same as before entering to the sleep mode.	
Caveats:	This service influences the functionality of the GPT channels. Therefore the mode switching of the Driver will be compatible to the overall state of the ECU.	
Configuration:	This function is pre-compile time configurable STD_ON / STD_OFF by the configuration parameter GptWakeupFunctionalityApi.	
DET:	GPT_E_UNINIT	Function called prior to Initialization function (Init function).
	GPT_E_PARAM_MODE	If function called with Invalid mode argument.
DEM:	None	
Safety Error reporting:	GPT_E_PARAM_MODE	If the parameter Mode is invalid
	GPT_E_NO_WAKEUP	If the function is called to SLEEP without

		IRQ enabled for any wakeup capable channel which is Started and wakeup enabled
Implementation Comments:	None	
I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is 'True' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'False'.	

6.2.11 Gpt_DisableWakeup

Table 71 Functional Specification - Gpt_DisableWakeup

Service name:	Gpt_DisableWakeup	
Syntax:	<pre>void Gpt_DisableWakeup (Gpt_ChannelType Channel)</pre>	
Source	This API is defined by the AUTOSAR.	
Service ID:	0x0A	
Sync/Async:	Synchronous	
Reentrancy:	Reentrant	
Parameters (in):	Channel	The associated Channel where the timer needs to be stopped
Parameters (out):	None	
Return value:	None	
Description:	This service disables the wakeup notification of the given GPT channel.	
Caveats:	Usage of Re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel.	
Configuration:	This function is pre-compile time configurable STD_ON / STD_OFF by the configuration parameter GptWakeupFunctionalityApi.	
DET:	GPT_E_UNINIT	If function is called prior to initialization (Init function).
	GPT_E_PARAM_CHANNEL	If function called with invalid argument channel number or non wakeup capable channel.
DEM:	None	
Safety Error reporting:	GPT_E_PARAM_CHANNEL	If the parameter Channel is invalid (not within the range specified by configuration) or if channel wakeup is not enabled by configuration (GptEnableWakeup)

Implementation Comments:	This API can be invoked when the CPU is in either User-1 Mode or User-0 Mode or Supervisor Mode.
I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is 'True' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'False'.

6.2.12 Gpt_EnableWakeup

Table 72 Functional Specification - Gpt_EnableWakeup

Service name:	Gpt_EnableWakeup	
Syntax:	<pre>void Gpt_EnableWakeup (Gpt_ChannelType Channel)</pre>	
Source	This API is defined by the AUTOSAR.	
Service ID:	0x0B	
Sync/Async:	Synchronous	
Reentrancy:	Reentrant	
Parameters (in):	Channel	The associated Channel where the timer needs to be stopped
Parameters (out):	None	
Return value:	None	
Description:	This service enables the wakeup notification of the given GPT channel.	
Caveats:	Usage of Re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel.	
Configuration:	This function is pre-compile time configurable STD_ON / STD_OFF by the configuration parameter GptWakeupFunctionalityApi.	
DET:	GPT_E_UNINIT	If function is called prior to initialization (Init function).
	GPT_E_PARAM_CHANNEL	If function called with invalid argument channel number or non wakeup capable channel.
DEM:	None	
Safety Error reporting:	GPT_E_PARAM_CHANNEL	If the parameter Channel is invalid (not within the range specified by configuration) or if channel wakeup is not enabled by configuration (GptEnableWakeup)
Implementation Comments:	None	

I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is 'True' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeRuntimeApiEnable is 'False'.
-----------------	--

6.2.13 Gpt_CheckWakeup

Table 73 Functional Specification - Gpt_CheckWakeup

Service name:	Gpt_CheckWakeup	
Syntax:	<pre>void Gpt_CheckWakeup (EcuM_WakeupSourceType wakeupSource)</pre>	
Source	This API is defined by the AUTOSAR.	
Service ID:	0x0C	
Sync/Async:	Synchronous	
Reentrancy:	Reentrant	
Parameters (in):	wakeupSource	Channel wakeup info.
Parameters (out):	None	
Return value:	None	
Description:	Checks if a wakeup capable GPT channel is the source for a wakeup event and calls the ECU state manager service EcuM_SetWakeupEvent in case of a valid GPT channel wakeup event.	
Caveats:	Usage of Re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel.	
Configuration:	This function is pre-compile time configurable STD_ON / STD_OFF by the configuration parameter GptWakeupFunctionalityApi.	
DET:	GPT_E_UNINIT	If function is called prior to initialization (Init function).
DEM:	None	
Implementation Comments:	None	
I/O Mode	This API can be invoked when the CPU is in either User-1 Mode or User-0 Mode or Supervisor Mode.	

6.2.14 Gpt_InitCheck

Table 74 Specification of API Gpt_InitCheck

Service name:	Gpt_InitCheck
----------------------	---------------

Syntax:	Std_ReturnType Gpt_InitCheck (const Gpt_ConfigType *ConfigPtr) 	
Source	This is a vendor defined API.	
Service ID:	NA	
Sync/ Async:	Synchronous	
Re-entrancy:	Non-reentrant	
Parameters (in):	ConfigPtr	Pointer to the configuration structure
Parameters (out):	None	
Return value:	E_OK – Initialization comparison successful E_NOT_OK – Initialization not properly done by GPT driver	
Description:	This routine verifies the initialization done by the GPT driver. This should be called after Gpt_Init().	
Caveats:	None.	
Configuration:	This service is enabled only when the preprocessor switch GPT_SAFETY_ENABLE and GPT_INITCHECK_API is defined to STD_ON.	
DET:	None	
DEM:	None	
Safety Error reporting	None	
Implementation Comments	To reduce code complexity, each comparison will be done by a XOR of value against the inverted value of the intended value or vice versa. The comparison result should always be 0xFFFFFFFF (all 1s). This will also improve runtime since multiple checks are avoided if no failures exist. There is a very less probability of failure happening and can be reported well within the diagnostic test interval; hence this decision.	
Design Decisions:	None	
I/O Mode	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter GptRunningInUser0Mode is 'True' and GptUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter GptRunningInUser0Mode is 'False' and GptUserModeInitApiEnable is 'False'.	

6.2.15 Gpt_GetMode

Table 75 Specification of API Gpt_GetMode

Service name:	Gpt_GetMode
Syntax:	Gpt_ModeType Gpt_GetMode(void)
Source	This is a vendor defined API.
Service ID:	NA
Sync/ Async:	Synchronous

Re-entrancy:	Reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	Mode of the GPT driver
Description:	This service returns the mode of the GPT driver.
Caveats:	None.
Configuration:	This service is enabled only when the preprocessor switch GPT_SAFETY_ENABLE and GPT_GETMODE_API is defined to STD_ON.
DET:	None
DEM:	None
Safety Error reporting	None
Implementation Comments	None
Design Decisions:	None
I/O Mode	This API can be invoked when the CPU is in either User-1 Mode or User-0 Mode or Supervisor Mode.

6.3 Interrupt Handling

The interrupt service routine in GPT Driver is used for notification, reporting wakeup event and calculating next timeout values for continuous mode channels.

6.3.1 Gpt_Isr

Table 76 GPT Interrupt Function

Service name:	Gpt_Isr	
Syntax:	void Gpt_Isr (uint8 ChannelNo)	
Service ID:	0x0D	
Sync/ Async:	Synchronous	
Re-entrancy:	Reentrant	
Parameters (in):	ChannelNo	GPT Channel Number
Parameters (out):	None	
Return value:	None	
Description:	ISR does the following • Call of Notification • Call of Wakeup Events, EcuM_CheckWakeup will be called for wakeup event. • In case the channel is OSM the channel status is set to expired.	
Caveats:	None	
Configuration:	GptReportWakeupSource should be STD_ON if wakeup event need to be called from Interrupt routine.	
DET:	None	
DEM:	None	
Safety Error reporting:	GPT_E_PARAM_CHANNEL	If a GPT channel number which is not configured is passed by Gtm_IsrTomModule/ Gtm_IsrAtomModule as input.

Implementation Comments:	None
I/O Mode	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.

General Notes:

There are 40 SRNs (Service Request Nodes) to service GTM TOM Channel interrupt requests

There are 36 SRNs to service GTM ATOM Channel interrupt requests

- Each SRN should be provided a valid priority in the supplied Gtm_Irq.c and make sure that the corresponding Interrupt sources are enabled.
- The following table shows the SRN Number and the associated TOM/ATOM Channel resource.

Table 77 Service Request Node – Hardware Resource GTM

	Hardware Resource	SRN Number
TOM0	CHANNEL0, CHANNEL1	SRC_GTMTOM00
	CHANNEL2, CHANNEL3	SRC_GTMTOM01
	CHANNEL4, CHANNEL5	SRC_GTMTOM02
	CHANNEL6, CHANNEL7	SRC_GTMTOM03
	CHANNEL8, CHANNEL9	SRC_GTMTOM04
	CHANNEL10, CHANNEL11	SRC_GTMTOM05
	CHANNEL12, CHANNEL13	SRC_GTMTOM06
	CHANNEL14, CHANNEL15	SRC_GTMTOM07
TOM1	CHANNEL0, CHANNEL1	SRC_GTMTOM10
	CHANNEL2, CHANNEL3	SRC_GTMTOM11
	CHANNEL4, CHANNEL5	SRC_GTMTOM12
	CHANNEL6, CHANNEL7	SRC_GTMTOM13
	CHANNEL8, CHANNEL9	SRC_GTMTOM14
	CHANNEL10, CHANNEL11	SRC_GTMTOM15
	CHANNEL12, CHANNEL13	SRC_GTMTOM16
	CHANNEL14, CHANNEL15	SRC_GTMTOM17
TOM2	CHANNEL0, CHANNEL1	SRC_GTMTOM20
	CHANNEL2, CHANNEL3	SRC_GTMTOM21
	CHANNEL4, CHANNEL5	SRC_GTMTOM22
	CHANNEL6, CHANNEL7	SRC_GTMTOM23
	CHANNEL8, CHANNEL9	SRC_GTMTOM24
	CHANNEL10, CHANNEL11	SRC_GTMTOM25
	CHANNEL12, CHANNEL13	SRC_GTMTOM26
	CHANNEL14, CHANNEL15	SRC_GTMTOM27
TOM3	CHANNEL0, CHANNEL1	SRC_GTMTOM30
	CHANNEL2, CHANNEL3	SRC_GTMTOM31
	CHANNEL4, CHANNEL5	SRC_GTMTOM32
	CHANNEL6, CHANNEL7	SRC_GTMTOM33
	CHANNEL8, CHANNEL9	SRC_GTMTOM34
	CHANNEL10, CHANNEL11	SRC_GTMTOM35

Hardware Resource		SRN Number
	CHANNEL12, CHANNEL13	SRC_GTMTOM36
	CHANNEL14, CHANNEL15	SRC_GTMTOM37
TOM4	CHANNEL0, CHANNEL1	SRC_GTMTOM40
	CHANNEL2, CHANNEL3	SRC_GTMTOM41
	CHANNEL4, CHANNEL5	SRC_GTMTOM42
	CHANNEL6, CHANNEL7	SRC_GTMTOM43
	CHANNEL8, CHANNEL9	SRC_GTMTOM44
	CHANNEL10, CHANNEL11	SRC_GTMTOM45
	CHANNEL12, CHANNEL13	SRC_GTMTOM46
	CHANNEL14, CHANNEL15	SRC_GTMTOM47
ATOM0	CHANNEL0, CHANNEL1	SRC_GTMATOM00
	CHANNEL2, CHANNEL3	SRC_GTMATOM01
	CHANNEL4, CHANNEL5	SRC_GTMATOM02
	CHANNEL6, CHANNEL7	SRC_GTMATOM03
ATOM1	CHANNEL0, CHANNEL1	SRC_GTMATOM10
	CHANNEL2, CHANNEL3	SRC_GTMATOM11
	CHANNEL4, CHANNEL5	SRC_GTMATOM12
	CHANNEL6, CHANNEL7	SRC_GTMATOM13
ATOM2	CHANNEL0, CHANNEL1	SRC_GTMATOM20
	CHANNEL2, CHANNEL3	SRC_GTMATOM21
	CHANNEL4, CHANNEL5	SRC_GTMATOM22
	CHANNEL6, CHANNEL7	SRC_GTMATOM23
ATOM3	CHANNEL0, CHANNEL1	SRC_GTMATOM30
	CHANNEL2, CHANNEL3	SRC_GTMATOM31
	CHANNEL4, CHANNEL5	SRC_GTMATOM32
	CHANNEL6, CHANNEL7	SRC_GTMATOM33
ATOM4	CHANNEL0, CHANNEL1	SRC_GTMATOM40
	CHANNEL2, CHANNEL3	SRC_GTMATOM41
	CHANNEL4, CHANNEL5	SRC_GTMATOM42
	CHANNEL6, CHANNEL7	SRC_GTMATOM43
ATOM5	CHANNEL0, CHANNEL1	SRC_GTMATOM50
	CHANNEL2, CHANNEL3	SRC_GTMATOM51
	CHANNEL4, CHANNEL5	SRC_GTMATOM52
	CHANNEL6, CHANNEL7	SRC_GTMATOM53
ATOM6	CHANNEL0, CHANNEL1	SRC_GTMATOM60
	CHANNEL2, CHANNEL3	SRC_GTMATOM61
	CHANNEL4, CHANNEL5	SRC_GTMATOM62
	CHANNEL6, CHANNEL7	SRC_GTMATOM63

Hardware Resource		SRN Number
ATOM7	CHANNEL0, CHANNEL1	SRC_GTMTATOM70
	CHANNEL2, CHANNEL3	SRC_GTMTATOM71
	CHANNEL4, CHANNEL5	SRC_GTMTATOM72
	CHANNEL6, CHANNEL7	SRC_GTMTATOM73
ATOM8	CHANNEL0, CHANNEL1	SRC_GTMTATOM80
	CHANNEL2, CHANNEL3	SRC_GTMTATOM81
	CHANNEL4, CHANNEL5	SRC_GTMTATOM82
	CHANNEL6, CHANNEL7	SRC_GTMTATOM83

6.4 Exclusive Area (EA) Information

No Exclusive area is provided for GPT.

7 Data Description

The below table lists all the global and static variables used in the Gpt Driver

Table 78 Global Data Description

Parameter	Type/Range	Description
Gpt_kConfigPtr	Gpt_ConfigType	This is a Pointer to GPT Driver configuration structure. Note: When the GptPBFixedAddress is ON, this will be initialized with &Gpt_ConfigRoot[0]
Gpt_DriverMode	uint8	Variable to store Driver current mode (SLEEP/ NORMAL)
Gpt_ChannelIndex	UInt8[]	Variable to store channel indices of all channels
Gpt_AsilFlag	UInt8[]	Variable to store Notification Flag and WakeUp Flag for all ASIL channels
Gpt_QMFlag	UInt8[]	Variable to store Notification Flag and WakeUp Flag for all QM channels
Gpt_AsilChannelStatus	UInt8 []	Variable to store channel status for ASIL channels
Gpt_QMChannelStatus	UInt8 []	Variable to store channel status for QM channels
Gpt_ChannelIdentifier	Gpt_ChannelIdentifierType	Variable to store GTM channel information corresponding to every GPT channel

8 Error Classification

Table 79 Error Classification

Error Code	Error Description	Relevance	Value (hex)
GPT_E_ALREADY_INITIALIZED	Function Gpt_Init called again without calling	Development	0x0D

Error Code	Error Description	Relevance	Value (hex)
	Gpt_DelNit		
GPT_E_UNINIT	Function (GetTimeElapsed/GetTimeRemaining/Start/StopTimer, Enable/DisableNotification, Enable/DisableWakeup, set mode, GetVersionInfo) or Gpt_DelNit called prior to init function	Development	0x0A
GPT_E_BUSY	Function 'Start Timer' or 'DelNit' called while timer is already running	Development	0x0B
GPT_E_PARAM_CHANNEL	Function (GetTimeElapsed/GetTimeRemaining/Start/StopTimer, Enable/DisableNotification, Enable/DisableWakeup) called with invalid channel ID. Enable/DisableWakeup called on a non-wakeup capable channel.	Development and Safety error	0x14
GPT_E_PARAM_VALUE	Function 'StartTimer' called with invalid value	Development and Safety error	0x15
GPT_E_PARAM_MODE	Gpt_SetMode called with invalid mode parameter	Development and Safety error	0x1F
GPT_E_PARAM_POINTER	Function 'Gpt_GetVersionInfo' and Gpt_Init is called with invalid pointer	Development and Safety error	0x16
GPT_E_NO_WAKEUP	'Gpt_SetMode' called to SLEEP without IRQ enabled for the wakeup channel which is Started and for which WakeUp is enabled.	Safety error	0x20

9 Example usage

This chapter describes how the GPT Driver can be configured and how to use different API provided by GPT Driver in the application.

9.1 Configuration of the Driver

To configure GPT Driver the following guidelines should be followed properly.

1. MCU Driver: Configure the System Clock
2. GTM Driver: Configure GTM Clock.
3. Interrupt Priority Initialization: The priority of interrupts should be configured in IRQ Driver and appropriate priorities should be given. The TOM/ATOM channel Interrupt map is given in [Table 77](#).
4. GPT Driver: Select the required API configuration switches, wakeup capability, notification and different types of timers used or not.

9.1.1 Sample Configuration of the Driver

Requirement: channel configuration consisting of each timer as follows.

Each Channel configuration needs GptAssignedHwUnit parameter to be configured with the proper channel. Refer [4.5.1.8](#) for configuration of the same and refer [2.1.2](#) for configuring different Timer Type selection.

1. Channel Name: Gpt_Channel0

Wakeup Capability: TRUE

Wakeup Info: 0

Assigned HW: Channel 0 of ATOM0

GptSignalType: GPT_ASIL_SIGNAL

Notification: Gpt_Notification

Mode: GPT_CH_MODE_CONTINUOUS

Sample configuration of Gtm TOM channel.

TomChannelUsage :

TomChannelUsage : USED_BY_OTHER_DRIVERS_OR_UNUSED

TomChannelTimerParameters :

TomChannelClockSelect : GTM_FIXED_CLOCK_0

TomChannelInterrupt :

TomNotification : Gpt_Isr

Sample configuration of Gtm ATOM channel.

AtomChannelUsage :

AtomChannelUsage : USED_BY_OTHER_DRIVERS_OR_UNUSED

AtomChannelTimerParameters :

AtomChannelClockSelect : GTM_CONFIGURABLE_CLOCK_0

AtomChannelInterrupt :

AtomNotification : Gpt_Isr

The header file is shown in [Code Listing 1](#). Only the important settings are listed below

Code Listing 1 Gpt Configuration Header file – Gpt_Cfg.h

```
001:  #define GPT_DEV_ERROR_DETECT                (STD_OFF)
002:  #define GPT_REPORT_WAKEUP_SOURCE            (STD_ON)
003:  #define GPT_DEINIT_API                      (STD_ON)
004:  #define GPT_VERSION_INFO_API                (STD_ON)
005:  #define GPT_TIME_ELAPSED_API                (STD_ON)
006:  #define GPT_TIME_REMAINING_API              (STD_ON)
007:  #define GPT_ENABLE_DISABLE_NOTIFICATION_API (STD_ON)
008:  #define GPT_WAKEUP_FUNCTIONALITY_API        (STD_ON)
009:  #define GPT_SAFETY_ENABLE                   (STD_ON)
010:  #define GPT_INITCHECK_API                   (STD_ON)
011:  #define GPT_GETMODE_API                     (STD_ON)
012:  #define GPT_RUNNING_IN_USER_0_MODE_ENABLE   (STD_OFF)
013:  #define GPT_USER_MODE_INIT_API_ENABLE       (STD_OFF)
```

Code Listing 1 Gpt Configuration Header file – Gpt_Cfg.h

```
014:  #define GPT_USER_MODE_DEINIT_API_ENABLE      (STD_OFF)
015:  #define GPT_USER_MODE_RUNTIME_API_ENABLE      (STD_OFF)
```

Code Listing 2 Gpt Configuration Source file – Gpt_PBCfg.c

```
016:  /*
017:    Channel Configuration Instance GptChannelConfigSet_0
018:  */
019:  const Gpt_ChannelConfigType GptChannelConfigSet_0[ ] =
020:  {
021:    /* Channel Symbolic Name(ChannelId) : Gpt_Channel0
022:      TimerType : GTM_ATOM0_CHANNEL0 in GPT_MODE_CONTINUOUS
023:    */
024:    {
025:      #if (GPT_ENABLE_DISABLE_NOTIFICATION_API == STD_ON)
026:        (Gpt_NotificationPtrType)Gpt_Notification, /* Notification Function */
027:      #endif
028:      #if (GPT_WAKEUP_FUNCTIONALITY_API == STD_ON)
029:        #if (GPT_REPORT_WAKEUP_SOURCE == STD_ON)
030:          0, /* Wakeup Info */
031:        #endif
032:        TRUE, /* Wakeup Capability */
033:      #endif
034:      /* Assigned Hardware Unit */
035:      GptAssignedHwUnit(GTM_ATOM0_CHANNEL0),
036:      GPT_MODE_CONTINUOUS /* Channel Mode */
037:      #if (GPT_SAFETY_ENABLE == STD_ON)
038:      GPT_ASIL_SIGNAL, /* Channel Signal Type */
039:      #endif
040:      GTM_FIXED_CLOCK_0 /* Channel Clock frequency */
041:    };
```

9.1.1.1 Driver Initialization

The [Code Listing 3](#) depicts the steps involved in the initialization of the GPT Driver. Note: User must take care to configure GPT Driver Initialization in EcuM init list. Also make sure that Mcu_Init is called before Gpt_Init.

Code Listing 3 Driver Initialization in EcuM_Init function

```
042:  /*Initialize MCU Driver*/
043:  Mcu_Init(&Mcu_ConfigRoot[0]);
044:  /*Call Driver Init One. Initialize drivers before OS Init*/
045:  EcuM_AL_DriverInitOne();
046:  #ifdef ECUM_USES_GPT
047:  /*Initialize GPT Driver*/
```

Code Listing 3 Driver Initialization in EcuM_Init function

```
048:    Gpt_Init(&Gpt_ConfigRoot[0]);  
049:    #endif  
050:    Gpt_InitCheck(&Gpt_ConfigRoot[0]);
```

9.1.1.2 API Usages**9.1.2 Starting a Timer Channel**

The Gpt Driver provides API to start timer channel. The API is called with parameters Channel and Timeout period value.

Code Listing 4 Starting a Channel

```
051:    Gpt_StartTimer(Gpt_Channel0, Gpt_ValueType(2000));  
052:    /* Gpt_Channel0 is the channel ID, Timeout value is given in ticks */
```

In the code listing channel Gpt_Channel0 is started with timeout value of 2000 ticks.

9.1.3 Stopping a Timer Channel

The Gpt Driver provides API to stop timer channel. The API is called with parameter as Channel.

Code Listing 5 Stopping a Channel

```
053:    Gpt_StopTimer(Gpt_Channel0);
```

In the code listing channel Gpt_ChannelFrgCon2 is stopped. Stopping a timer channel already stopped or not started drives the API to report a DET error, in production mode there is no effect.

9.1.4 Enable Notification for a Timer Channel

The Gpt Driver provides API to enable notification for a timer channel. When enabled before the timer channel expires, results in user defined notification function to be called out by the GPT Driver when the timer expires.

This API is available with GptEnableDisableNotificationApi is switched STD_ON.

Code Listing 6 Enabling Notification For a Timer Channel

```
054:    Gpt_EnableNotification(Gpt_Channel0);
```

Notification can be enabled before starting a timer channel or during runtime.

9.1.5 Disable Notification for a Timer Channel

The Gpt Driver provides API to disable notification for a timer channel. When disabled before the timer channel expires, the user defined notification function is not called out by the GPT Driver when the timer expires.

This API is available with GptEnableDisableNotificationApi is switched STD_ON.

Code Listing 7 Disabling Notification For a Timer Channel

```
055:    Gpt_DisableNotification(Gpt_Channel0);
```

Notification can be disabled before starting a timer channel or during runtime. By default the notification is disabled after initialization.

9.1.6 To Get Time Elapsed For a Timer Channel

The Gpt Driver provides API to get time elapsed on a timer channel which is already started.

This API is available with GptTimeElapsedApi is switched STD_ON.

Code Listing 8 To Get Time Elapsed For a Timer Channel

```
056: ElapsedValue = Gpt_GetTimeElapsed(Gpt_Channel0);
```

9.1.7 To Get Time Remaining For a Timer Channel

The Gpt Driver provides API to get time remaining on a timer channel which is already started.

This API is available with GptTimeRemainingApi is switched STD_ON.

Code Listing 9 To Get Time Remaining For a Timer Channel

```
057: RemainingValue = Gpt_GetTimeRemaining(Gpt_Channel0);
```

9.1.8 To set GPT driver To Sleep Mode

The Gpt Driver provides API to set GPT driver to Sleep Mode.

Code Listing 10 Set GPT Driver to SLEEP Mode.

```
058: Gpt_SetMode(GPT_MODE_SLEEP);
```

In this mode only wakeup capable channels are active or available. The Driver is set back to normal mode by calling the same API with parameter GPT_MODE_NORMAL.

9.1.9 To Enable/Disabled Wakeup

The Gpt Driver provides API to Enable/Disable Wakeup for a wakeup capable channel.

Code Listing 11 Enable/Disable Wakeup.

```
059: Gpt_EnableWakeup (Gpt_Channel0);  
060: Gpt_DisableWakeup (Gpt_Channel0);
```

Note: Enable/Disable of wakeup should be done before setting Driver mode to SLEEP, i.e. Gpt_SetMode (GPT_MODE_SLEEP) should be the last API service call to set Driver to sleep. After wakeup before any API service is called of GPT Driver, Gpt_SetMode (GPT_MODE_NORMAL) should be called.

These API's are available with GptWakeupFunctionalityApi is switched STD_ON and feasible if GptReportWakeupSource is STD_ON. And can be called on a wakeup capable TRUE configured channel.

9.1.10 Getting Version Information

This API Gpt_GetVersionInfo is available with the pre-compiler switch GptVersionInfoApi set to STD_ON.

Code Listing 12 Getting Version Information of GPT Driver

```
061: Gpt_GetVersionInfo(&VersionInfoPtr);
```

9.1.11 Deinitializing GPT Driver

Code Listing 13 Deinitializing GPT Driver

```
062: Gpt_DeInit();
```

The Driver is deinitialized completely for the configuration provided in the Driver configuration structure.

9.1.12 To get GPT driver Mode

This API `Gpt_GetMode` is available with the pre-compiler switches `GPT_GETMODE_API` and `GPT_SAFETY_ENABLE` set to `STD_ON`.

Code Listing 14 To get GPT driver Mode

```
063: Gpt_GetMode();
```

The API returns mode of GPT Driver (`GPT_MODE_UNINITIALIZED`, `GPT_MODE_NORMAL`, `GPT_MODE_SLEEP`).

10 Limitations and Assumptions

10.1 Assumptions and Deviations from Software Specification

Table 80 Assumptions

Sl. No	Reference	Assumptions
1	General	<code>Gpt_GetTimeElapsed</code> shall be called only after <code>Gpt_StartTimer</code> , to ensure that <code>Gpt_StartTimer</code> is done correctly.
2	General	Timer resolutions should take care of ISR performance time and also read time. If the timeout value of the intended duration is very low and if there is not enough time for ISR's to get completed then there is a higher possibility of loss of events / incorrect notifications.

Deviations:

- 1) `GptConf_GptChannel_GptKanal` is generated instead of `GptConf_GptChannelConfiguration_GptKanal` (as per `ecuc_sws_2108`).

The Following Bugzillas are considered in implementing Gpt Driver

1. 51410: `GPT_MODE_UNINITIALIZED` is included in `Gpt_ModeType`.

2. 46181: Dem_EventStatusType is added to imported types.
3. 24191: GPT Driver mode is set to GPT_MODE_NORMAL after Gpt_Init execution.
4. 17946: Req Ids GPT255, GPT157 & GPT158 are not considered for implementation.

10.2 Limitations

10.2.1 Interrupt Latency

During the notification, if another notification occurred on the same channel faster than the Interrupt latency, Driver will not provide missed notifications.

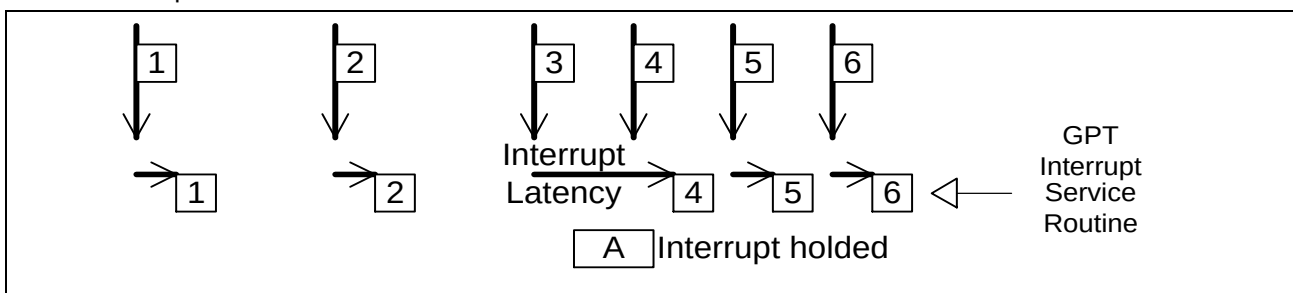


Figure 5 Software limitation in minimum time (high frequency) measurements

- In the above [Figure 5](#), it can be observed that, when notification 3 occurs, because of interrupt latency and before executing Gpt interrupt routine, the notification 4 occurs. In this scenario user will get notification only one time.
- Similarly the GPT software limits the minimum time duration measurement that GPT software can measure without giving arbitrary results.
- For each Gpt Channel Config Set, there has to be one corresponding Gtm Configuration/Mcu Module Config Set. So that any channel of a Gpt Config set shall use Gtm Tom or Atom Channels belongs to its corresponding Mcu Config Set only.
- Resource clash within a config set is handled by configuration tool. An error is asserted if the same Tom or Atom channel is being used by different Gpt channels within a config set.

10.3 Hardware Features That Are Not Supported

GPT Functionality on GPT12 hardware is not supported.

www.infineon.com